Machine Learning

Introduction au Machine Learning

Julien NIGOU

ELISA AEROSPACE

Elisa 5 — 2025/2026

Plan du cours (1/3)

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML
2.1 Jeu de données : features + labels
2.2 Nettoyage et préparation

Plan du cours (2/3)

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle
4.1 Métriques principales
4.2 Biais/variance

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

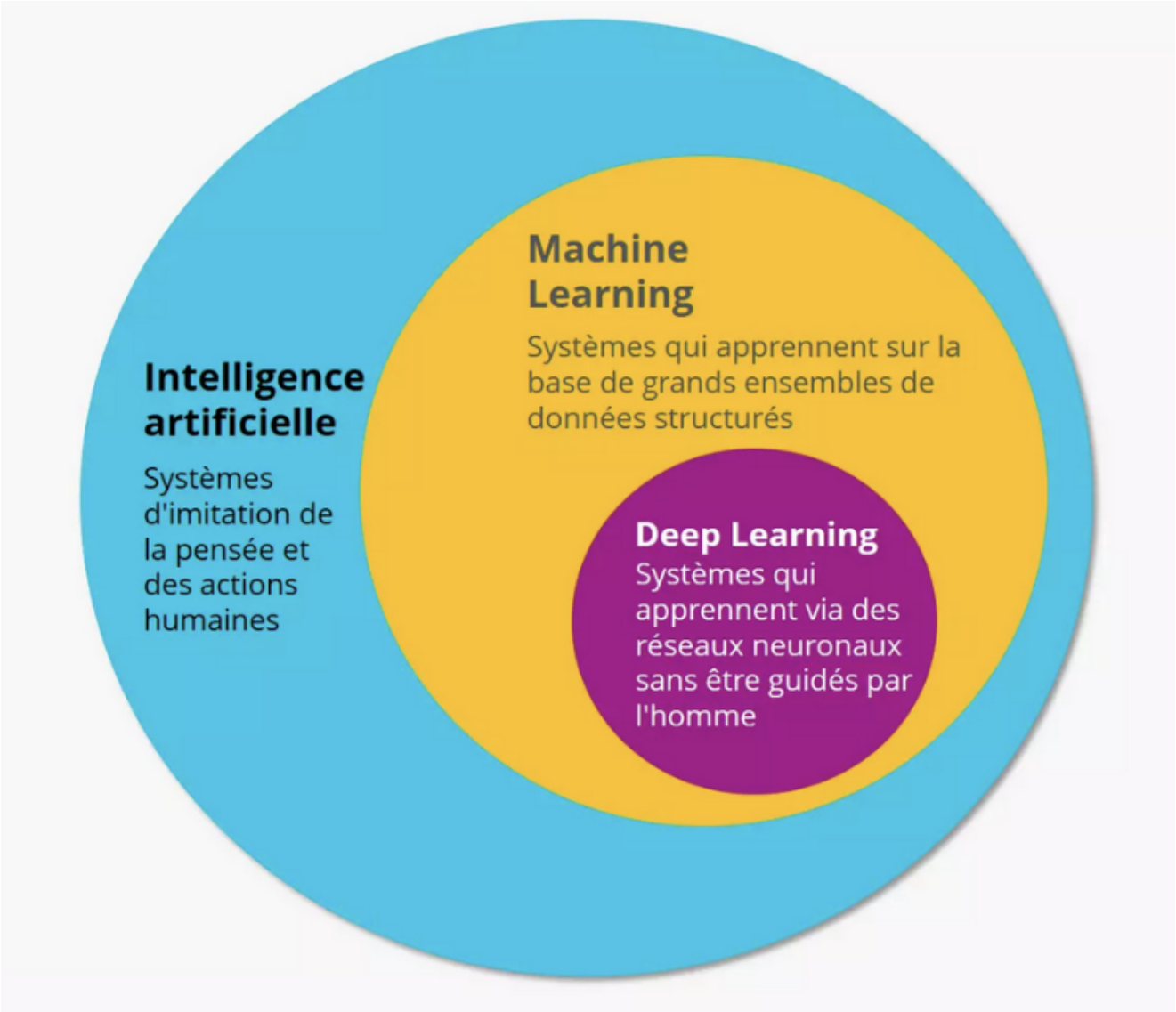
3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Videos

- Reinforcement learning
- Computer Vision - Car
- Computer Vision - Medical
- Live translation

Intelligence Artificielle



Qu'est-ce que le *Machine Learning* ?

- Sous-domaine de l'informatique qui construit des algorithmes s'appuyant sur des **exemples** d'un phénomène pour être utiles.
- Autre point de vue : résoudre un **problème pratique** en
  1. constituant un **jeu de données**, puis
  2. apprenant **algorithmiquement** un **modèle statistique** à partir de ces données.
- Ce modèle est ensuite utilisé pour **résoudre** le problème (prédire, classer, décider, etc.).

Exemples d'applications

- **Vision** : détection d'objets, diagnostic médical assisté.
- **Texte** : filtrage de spam, analyse de sentiments.
- **Prévision** : demande, séries temporelles, maintenance prédictive.
- **Recommandation** : contenus, produits, publicités.

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Panorama des types d'apprentissage

- **Supervisé**
- **Non supervisé**
- **Semi-supervisé**
- **Par renforcement**

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Apprentissage **supervisé** (Supervised Learning)

- Jeu de données labellisé : $\{(x_i, y_i)\}_{i=1}^{N}$.
- Ex. : si x décrit une personne : $x^{(1)}$ taille (cm), $x^{(2)}$ poids (kg), $x^{(3)}$ genre, ...
- Objectif : apprendre un **modèle** f qui, pour un x, fournit l'info permettant de **déduire** y (p. ex. probabilité de cancer).

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Apprentissage **non supervisé** (Unsupervised Learning)

- Jeu de données non labellisé : $\{x_i\}_{i=1}^{N}$.
- Objectif : apprendre un **modèle** qui transforme x en
  - un **identifiant de cluster** (clustering),
  - un **vecteur** de plus faible dimension (réduction dimensionnelle),
  - un **score d'anomalie** (détection d'outliers),
  - ou une valeur exploitable pour le problème pratique.

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Apprentissage **semi-supervisé** (Semi-Supervised Learning)

- Données mélangeant **labellisé** et **non labellisé** ; en général $|\mathcal{U}| \gg |\mathcal{L}|$.
- Objectif identique au **supervisé** : apprendre un modèle prédictif performant.
- Intuition : les nombreux exemples **non labellisés** révèlent la **structure** de la distribution des données, aidant à apprendre un meilleur modèle.

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Apprentissage **par renforcement** (Reinforcement Learning)

- Une **agent** interagit avec un **environnement** ; chaque état est perçu comme un vecteur de caractéristiques.
- À chaque état : l'agent choisit une **action** $\rightarrow$ reçoit une **récompense** et l'environnement peut **transitionner** vers un autre état.
- Objectif : apprendre une **politique** π qui, à partir de l'état, choisit l'action **optimale** maximisant la récompense **moyenne à long terme**.
- Cas d'usage : jeux, robotique, gestion de ressources, logistique.
- *Remarque de périmètre* : ici, on met l'accent sur les décisions **one-shot** (exemples i.i.d.) ; le RL n'est pas détaillé davantage.

Plan

1. Introduction
1.1 Qu'est-ce que le Machine Learning ?
1.2 Types d'apprentissage
1.3 Apprentissage supervisé
1.4 Apprentissage non supervisé
1.5 Apprentissage semi-supervisé
1.6 Apprentissage par renforcement
1.7 Comment fonctionne l'apprentissage supervisé

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Apprentissage supervisé — Vue d'ensemble

- Processus typique :
  1. **Collecter** et **labelliser** les données.
  2. **Représenter** chaque entrée par un **vecteur of characteristics** $x \in \mathbb{R}^D$.
  3. **Choisir** un algorithme et **entraîner** pour obtenir un modèle f.
  4. **Evaluer** le modèle.

Plan

1. Introduction

2. Les données : le cœur du ML
2.1 Jeu de données : features + labels
2.2 Nettoyage et préparation

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Plan

1. Introduction

2. Les données : le cœur du ML
2.1 Jeu de données : features + labels
2.2 Nettoyage et préparation

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

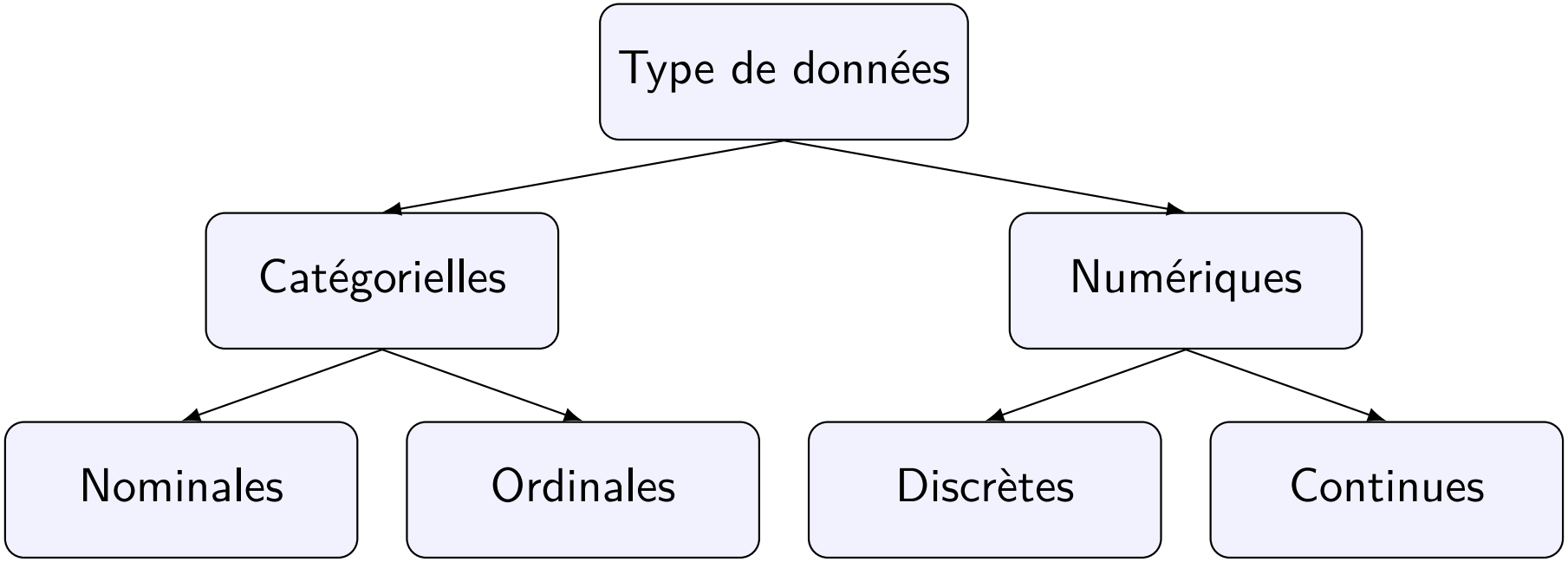
Jeu de données — définitions

- **Exemple / observation** : une entité du monde réel (email, image, patient, ...).
- **Vecteur de caractéristiques** : $x_i = (x_i^{(1)}, \ldots, x_i^{(D)}) \in \mathbb{R}^D$.
- **Label / cible** : y_i (classe, réel, structure).
- **Jeu de données supervisé** : $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$.
- **Forme** : $X \in \mathbb{R}^{N \times D}$, $y \in \mathbb{R}^N$ (ou $\{1, \ldots, C\}^N$).

Labels (cibles) & tâches

- **Régression** : $y \in \mathbb{R}$ (valeur continue).
- **Classification binaire** : $y \in \{0, 1\}$ ou $\{-1, +1\}$.
- **Multiclasse** : $y \in \{1, \ldots, C\}$ ou $\mathbf{Y} \in \{0, 1\}^{N \times C}$ (one-hot).
- **Multi-label** : plusieurs labels vraies par exemple.

Types de variables (features)



- **Textes :** documents, tokens, n-grammes, embeddings.
- **Temps & séries :** dates, saisonnalité, lags, rolling stats.
- **Images / audio :** pixels normalisés, spectrogrammes, features CNN.

Données catégorielles

Categorical Data

- Décrivent des caractéristiques qualitatives (ex. genre).
- Ne fournissent pas de valeur quantitative.
- Ne permettent pas d'opérations arithmétiques.

Nominal Data

- Pas d'ordre intrinsèque : ne peuvent pas être comparées.
- Exemples : genre, statut marital, couleur.

Ordinal Data

- Possèdent un ordre logique séquentiel.
- Exemples : taille de vêtements (S, M, L, XL...), satisfaction (faible, moyen, élevé), opinion (fortement en désaccord $\rightarrow$ fortement d'accord).

Données numériques

Numerical Data

- Exprimées sous forme de valeurs numériques (prix, taille. . .).
- Permettent des opérations arithmétiques et des analyses statistiques.

Discrete Data

- Peuvent prendre uniquement des valeurs finies (ou dénombrables).
- Exemples : nombre d'enfants, nombre de pièces dans une maison.

Continuous Data

- Peuvent prendre une infinité de valeurs (réelles).
- Exemples : poids, température, taux de chômage.

Exemple : représentation des données

Considérons les poids d'un groupe de 1000 personnes.

- Difficile de décrire les données simplement en regardant les nombres !

$\Longrightarrow$

```
[88 67 73 62 60 81 58 66 52 76 61 76 79 78 85 79 57 83 55 88 82 58 86 76
 76 89 90 71 77 87 50 84 63 63 75 65 78 67 86 79 79 49 72 63 74 62 83 63
 55 63 85 64 55 56 76 63 61 80 59 58 56 85 83 75 80 67 84 83 83 77 64 78
 52 54 65 67 64 78 59 79 62 76 55 61 58 79 65 85 58 80 82 62 68 65 80 79
 76 80 63 79 58 64 79 87 54 74 70 73 56 59 66 59 82 58 90 88 57 62 61 62
 59 78 60 58 81 47 55 78 76 80 61 66 59 80 76 69 62 66 83 79 57 76 63 60
 59 77 61 78 58 61 79 65 56 60 81 66 81 64 74 61 80 61 76 67 55 52 59 74
 68 72 76 75 74 60 54 60 89 75 75 86 67 61 59 66 82 59 78 60 87 56 76 92
 59 80 82 84 80 83 80 73 55 60 86 67 80 69 64 57 66 90 53 89 81 50 74 84
 59 65 71 55 61 81 60 79 71 52 58 57 82 54 67 52 63 78 58 82 77 57 80 90
 90 63 78 78 76 85 71 79 79 62 48 59 53 66 81 67 71 75 79 67 54 62 66 82
 49 59 84 64 64 74 86 82 85 72 76 85 95 76 79 48 57 54 90 72 83 81 55 82
 80 82 65 63 75 61 61 46 77 82 80 82 69 77 61 56 51 83 57 69 54 75 88 80
 59 64 54 48 63 56 80 87 55 79 55 67 60 83 77 81 51 83 60 56 86 81 83 61
 58 52 54 78 66 65 64 87 59 60 63 79 82 68 76 70 63 51 81 84 59 60 69 82
 60 61 61 62 64 78 82 84 82 78 62 83 84 79 63 78 64 75 68 62 89 57 72 85
 78 63 61 76 80 65 62 84 80 80 76 58 84 64 60 63 80 54 60 75 90 88 59 68
 75 80 57 60 70 64 81 75 80 82 58 84 65 57 83 54 85 58 55 59 59 84 85 49
 79 82 73 88 54 58 81 88 59 59 63 85 83 66 79 72 53 58 74 78 61 65 66 53
 61 80 59 74 55 80 90 50 81 62 84 59 59 57 75 55 63 81 58 78 83 59 83 63
 83 52 62 55 81 61 61 54 77 79 56 56 59 84 55 62 56 67 52 76 69 59 58 61
 83 70 51 59 55 77 63 60 88 58 85 64 76 54 78 61 77 64 85 59 61 82 77 58
 65 82 61 87 57 85 57 83 83 83 82 79 75 57 78 65 54 83 79 59 54 62 80 63
 84 62 56 64 74 85 63 57 86 54 78 80 84 57 65 55 77 61 60 85 79 60 52 78
 76 64 73 82 76 65 56 86 55 82 84 50 82 65 60 75 73 61 79 82 53 79 80 57
...
 68 78 69 84 68 65 83 59 57 55 73 79 61 59 59 78 84 83 59 59 81 84 88 59
 64 71 92 61 73 56 60 79 64 81 88 71 59 60 77 65 81 74 52 62 55 57 56 50
 83 82 56 83 67 84 74 66 57 60 60 53 79 86 55 87 86 79 57 57 76 80 74 56
 63 60 58 82 81 81 71 66 89 79 58 53 80 84 59 76]
```

Statistiques descriptives

- Résument ou décrivent les caractéristiques d'un jeu de données.
- Trois grandes catégories de mesures :
  - **Tendance centrale** : décrit le centre du jeu de données (moyenne, médiane, mode).
  - **Variabilité (ou dispersion)** : décrit la dispersion du jeu de données (variance, écart-type).
  - **Distribution de fréquence** : décrit l'occurrence des valeurs dans le jeu de données (effectif).

Statistiques

```
print(f"Mean: {weights_df.mode().values}")
print(f"Median: {weights_df.mean().values}")
print(f"Mode: {weights_df.mode().values}")
```

```
Mean: [[59]]
Median: [69.7]
Mode: [[59]]
```

FIGURE – Central tendencies

```
print(f"Standard deviation: {weights_df.std().values}")
print(f"Variance: {weights_df.var().values}")
```

```
Standard deviation: [11.21988796]
Variance: [125.88588589]
```

FIGURE – Variability

```
import seaborn as sns
sns.set_theme(style="ticks")

%matplotlib inline
ax = sns.histplot(weights_all)
_ = ax.set(xlabel='Weight (Kg)', ylabel='Count')
```

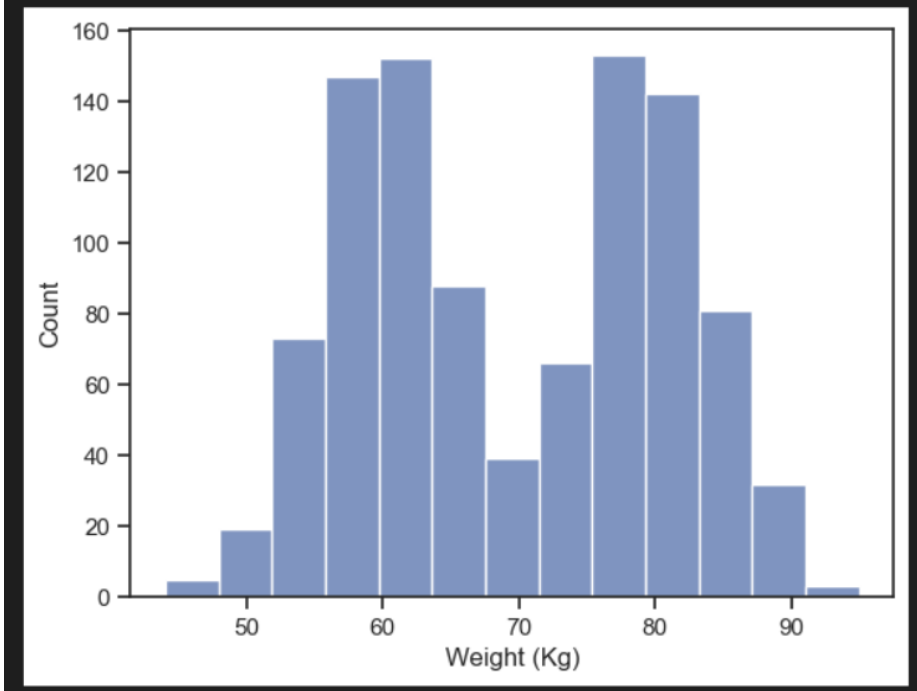
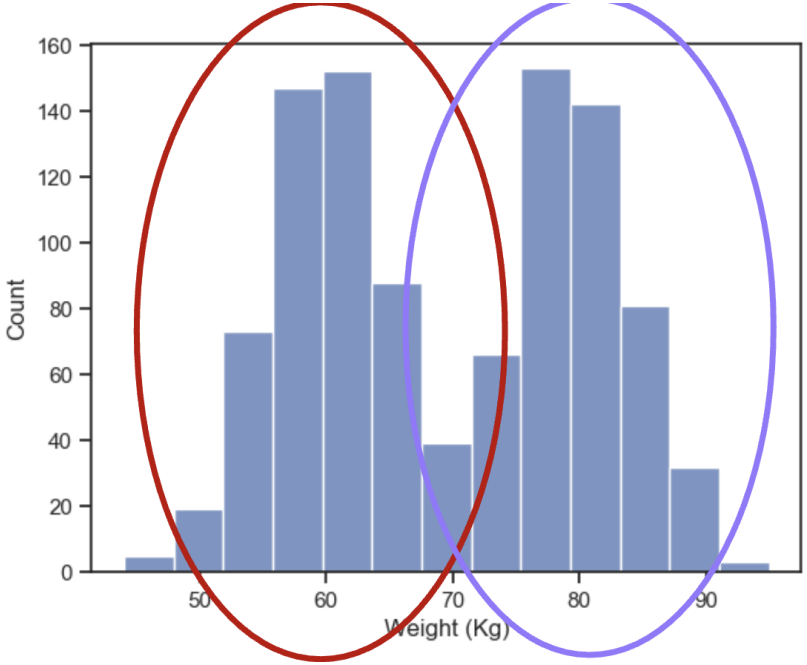


FIGURE – Frequency Distribution

Exemple : bimodal distribution (deux pics)

- **2 pics : Pourquoi ?**

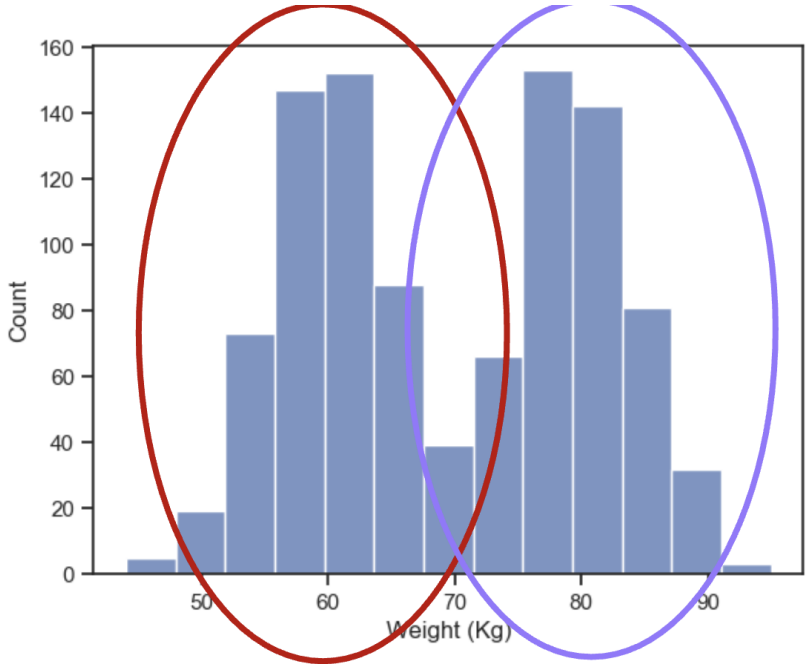


Exemple : bimodal distribution (deux pics)

- **2 pics : Pourquoi ?**
- Sous-populations : hommes et femmes.

Considérons une nouvelle personne dont le poids est inconnu.

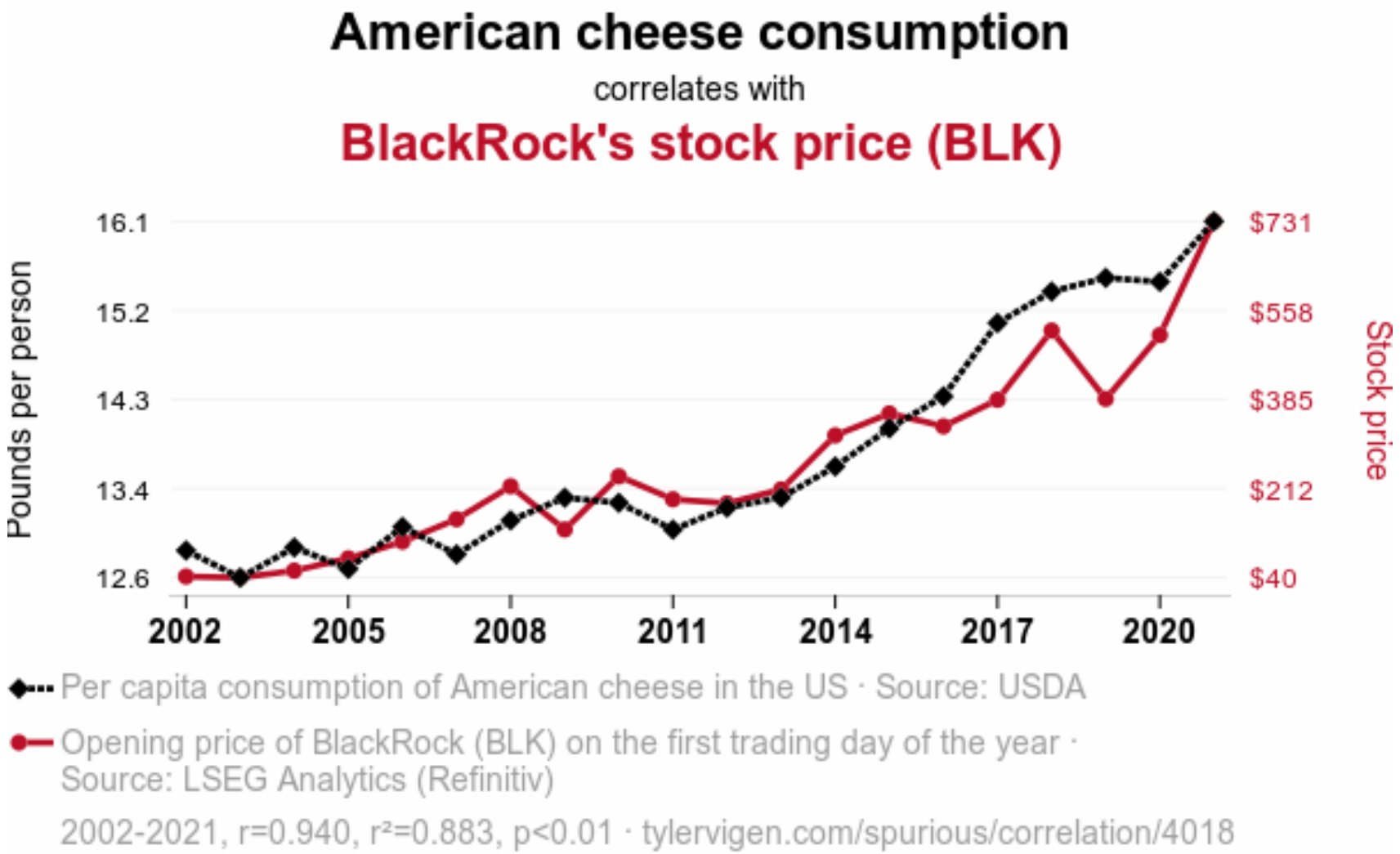
- Quel serait un prédicteur simple de son poids ?



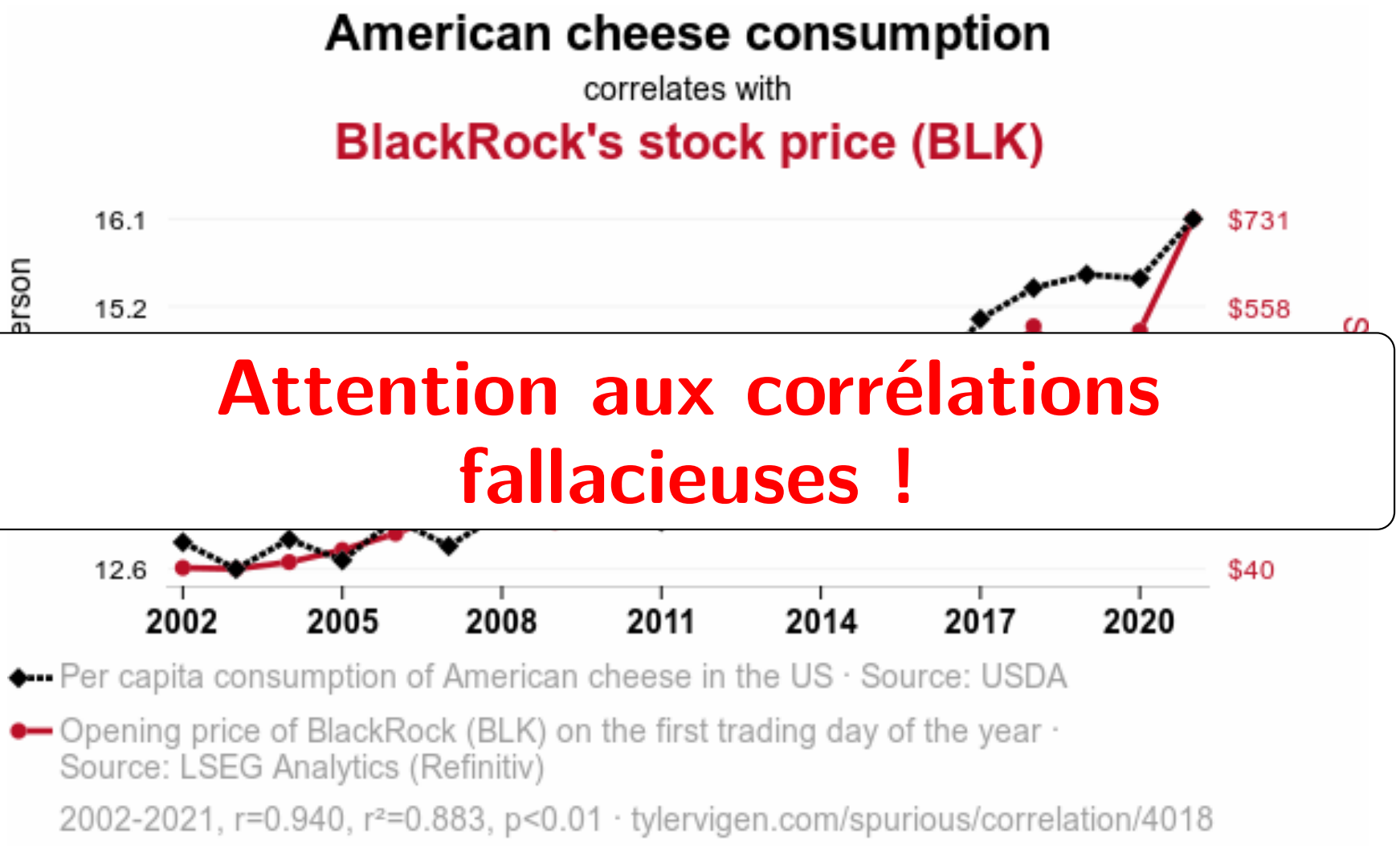
Statistiques inférentielles

Elles permettent de faire des prédictions (*inferences*) à partir des données, en exploitant les structures sous-jacentes dans un échantillon pour les généraliser à une population plus large.

Pattern


American cheese consumption
correlates with
BlackRock's stock price (BLK)
Pounds per person
16.1
15.2
14.3
13.4
12.6
$731
$558
$385
$212
$40
Stock price
2002
2005
2008
2011
2014
2017
2020
Per capita consumption of American cheese in the US · Source: USDA
Opening price of BlackRock (BLK) on the first trading day of the year · Source: LSEG Analytics (Refinitiv)
2002-2021, r=0.940, r²=0.883, p<0.01 · tylervigen.com/spurious/correlation/4018

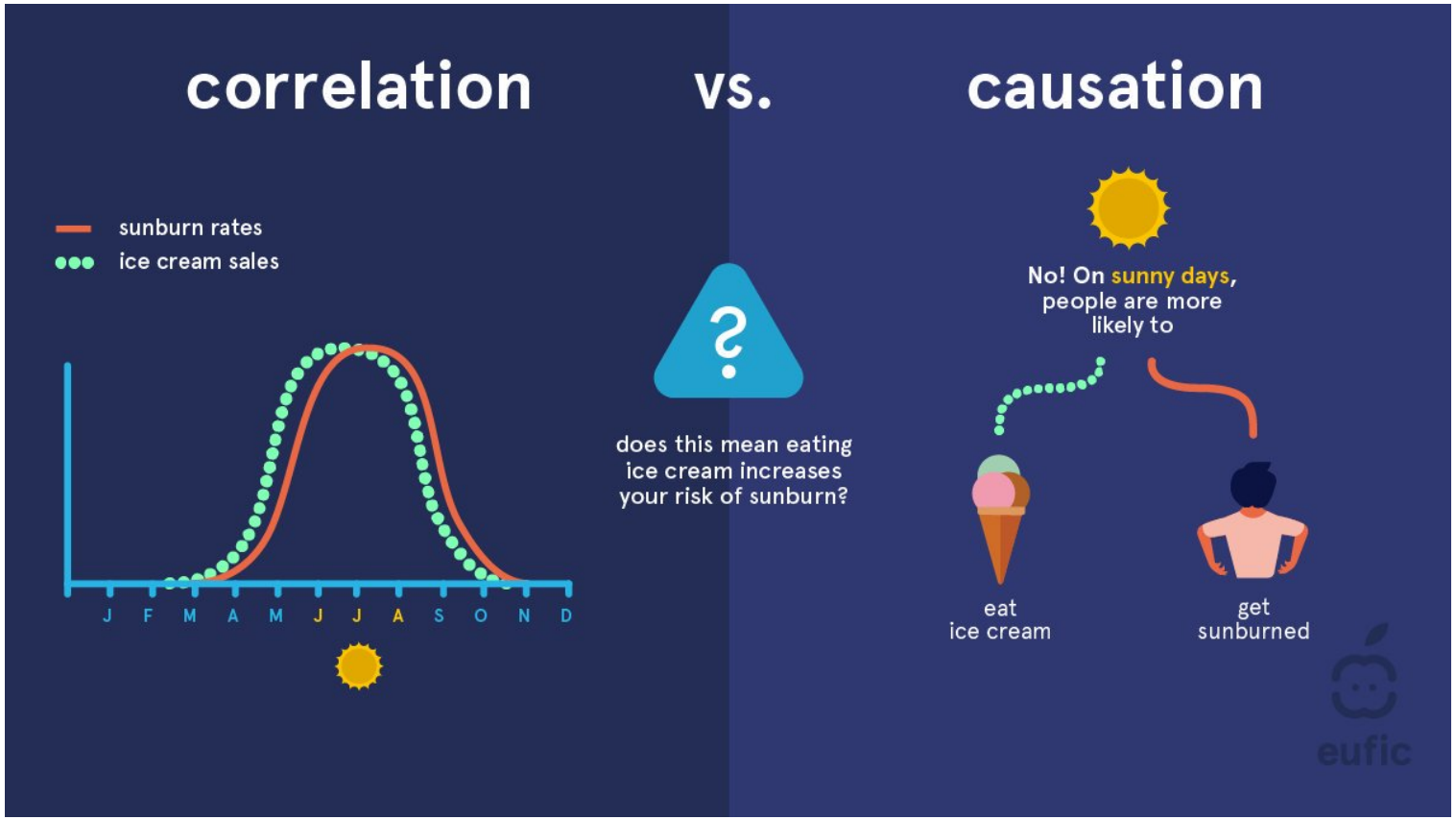
Pattern



Attention aux corrélations fallacieuses !

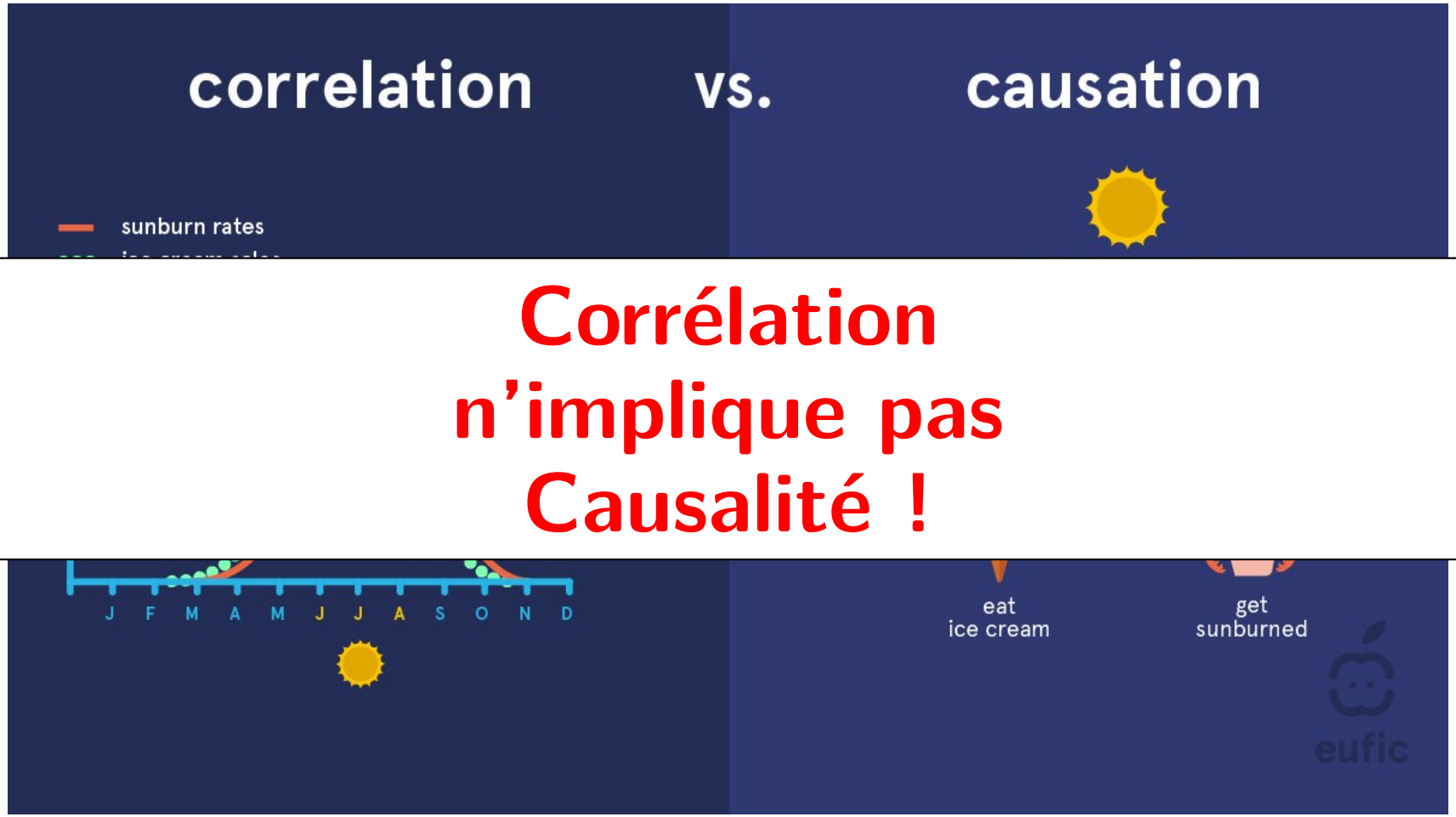
Corrélation n'est pas causalité

- Données : coups de soleil et consommation de glace
- Observation : plus on mange de glace, plus le risque de coup de soleil augmente. Est-ce que la glace provoque les coups de soleil ?



Source : European Food Information Council

Pattern



Corrélation n'implique pas Causalité !

Plan

1. Introduction

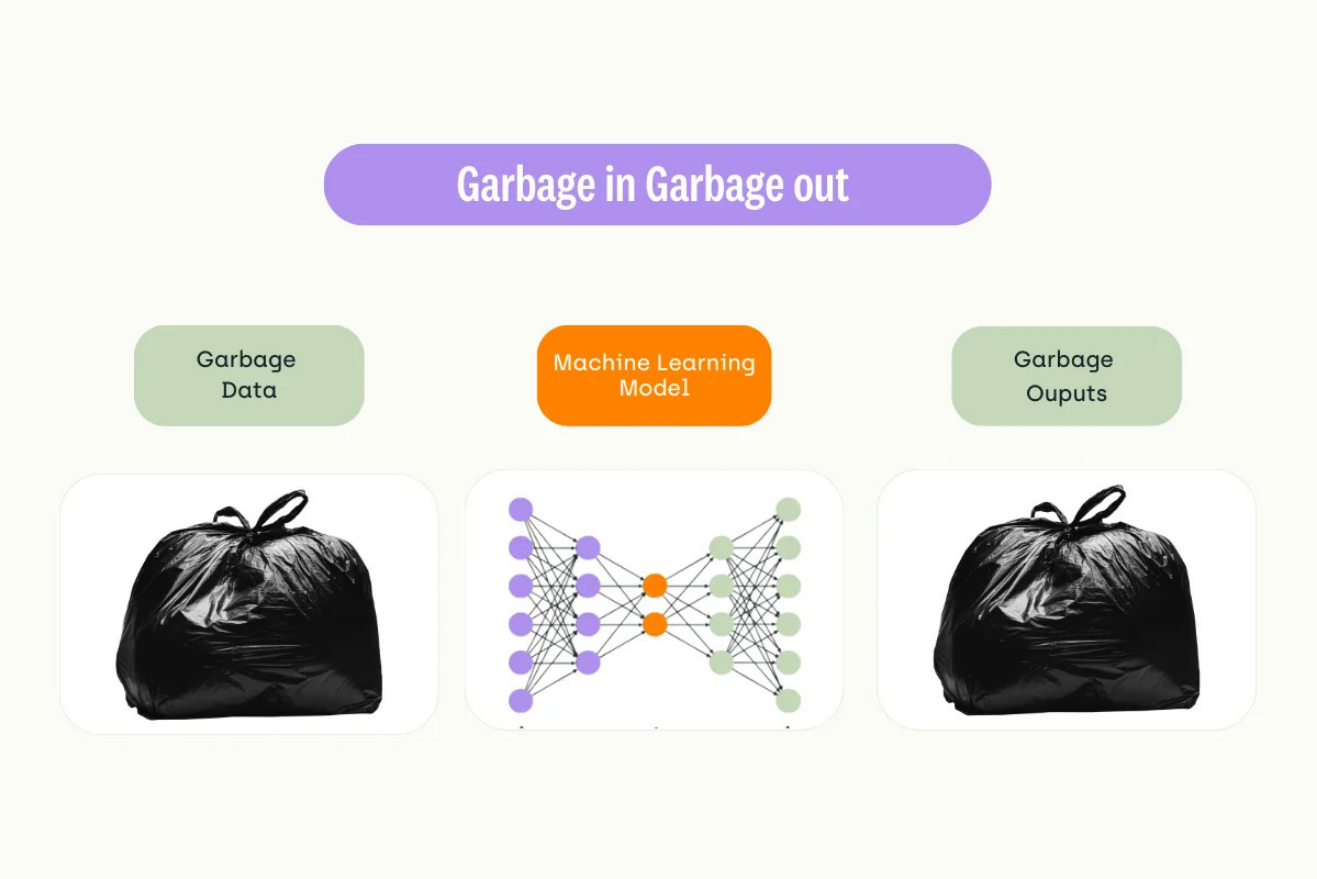
2. Les données : le cœur du ML
2.1 Jeu de données : features + labels
2.2 Nettoyage et préparation

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle

Qualité des données : problèmes courants

- **Valeurs manquantes**, duplicats, incohérences (unités, formats).
- **Outliers** / valeurs aberrantes, erreurs de saisie.
- **Caractères spéciaux**, espaces, encodage texte.
- **Données non stationnaires** (drift), échantillonnage biaisé.



Data Cleaning

Objectif : Corriger ou traiter les problèmes de qualité des données (valeurs manquantes, outliers, déséquilibre, incohérences).

Valeurs manquantes : stratégies

- **Supprimer** lignes/colonnes (si impact limité).
- **Imputation simple** : moyenne/médiane (numérique), mode (catégoriel).
- **Imputation avancée** : k-NN imputer, régressions itératives (MICE).
- **Indicateur de manquant** (flag binaire) pour capturer l'information.

Outliers, déséquilibre

- **Outliers** : détection (z-score, IQR, méthodes robustes), **robustifier** (winsorisation) si pertinent.
- **Déséquilibre** : *class weights*, *resampling* (SMOTE, undersampling), ajustement du seuil.

Data Transformation

Objectif : Transformer les variables pour les rendre exploitables par les modèles. (encodage, normalisation, mise à l'échelle)

Encodage des variables catégorielles

- **Ordinal encoding** : si ordre intrinsèque.
- **One-Hot encoding** : sinon (variables nominales).
- **Target encoding** : haute cardinalité (attention au **leakage** $\rightarrow$ utiliser CV).
- **Hashing trick** : très grands vocabulaires (textes, IDs).
- **Rare categories** : regrouper en « Autre ».

Mise à l'échelle des features

- **Standardisation** : $z = \dfrac{x - \mu}{\sigma}$ (utile pour SVM, k-NN, régressions, réseaux).
- **Min–Max scaling** : $\dfrac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$ (bornes [0,1]).
- **Robust scaling** : basé sur médiane & IQR (moins sensible aux outliers).

Data Reduction

Objectif : Réduire la dimensionnalité ou la taille des données pour accélérer l'entraînement et éviter le surapprentissage.

Méthodes de réduction

- **Feature selection** :
  - Basée sur la variance, les corrélations.
  - Méthodes wrapper (RFE).
  - Importance des features (arbres, modèles linéaires).
- **Feature extraction** :
  - PCA (réduction par variance expliquée).
  - t-SNE / UMAP (visualisation en 2D/3D).
  - Auto-encodeurs (deep learning).
- **Sampling** : réduction de la taille de l'échantillon.

Découpage des données

- **Split** : Train / Validation / Test.
- **Stratifier** en classification pour respecter les proportions de classes.
- **Pas de fuite de données** : tous les *fits* (impute, scaler, encodeur) se font **uniquement** sur **Train**, puis se *transforment* sur Val/Test.
- **Cross-validation** k-fold pour une estimation plus stable.

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle

Régression linéaire — idée

Qu'est-ce que la régression ?

Tâche de **supervised learning** qui prédit une **sortie continue** $y \in \mathbb{R}$ à partir de features d'entrée $x \in \mathbb{R}^D$.

Régression linéaire — idée

Qu'est-ce que la régression ?

Tâche de **supervised learning** qui prédit une **sortie continue** $y \in \mathbb{R}$ à partir de features d'entrée $x \in \mathbb{R}^D$.

Modèle de régression linéaire

$$\hat{y} = f_{w,b}(x) = w^\top x + b$$

Régression linéaire — idée

Qu'est-ce que la régression ?

Tâche de **supervised learning** qui prédit une **sortie continue** $y \in \mathbb{R}$ à partir de features d'entrée $x \in \mathbb{R}^D$.

Modèle de régression linéaire

$$\hat{y} = f_{w,b}(x) = w^\top x + b$$

Où :

- $\hat{y}$: sortie prédite.
- $x = [x^{(1)}, \ldots, x^{(D)}]$. : vecteur des features.
- $w = [w^{(1)}, \ldots, w^{(D)}]^\top$: poids (importance des features).
- $b \in \mathbb{R}$: biais (décalage global).

Régression linéaire en 1D (intuition)

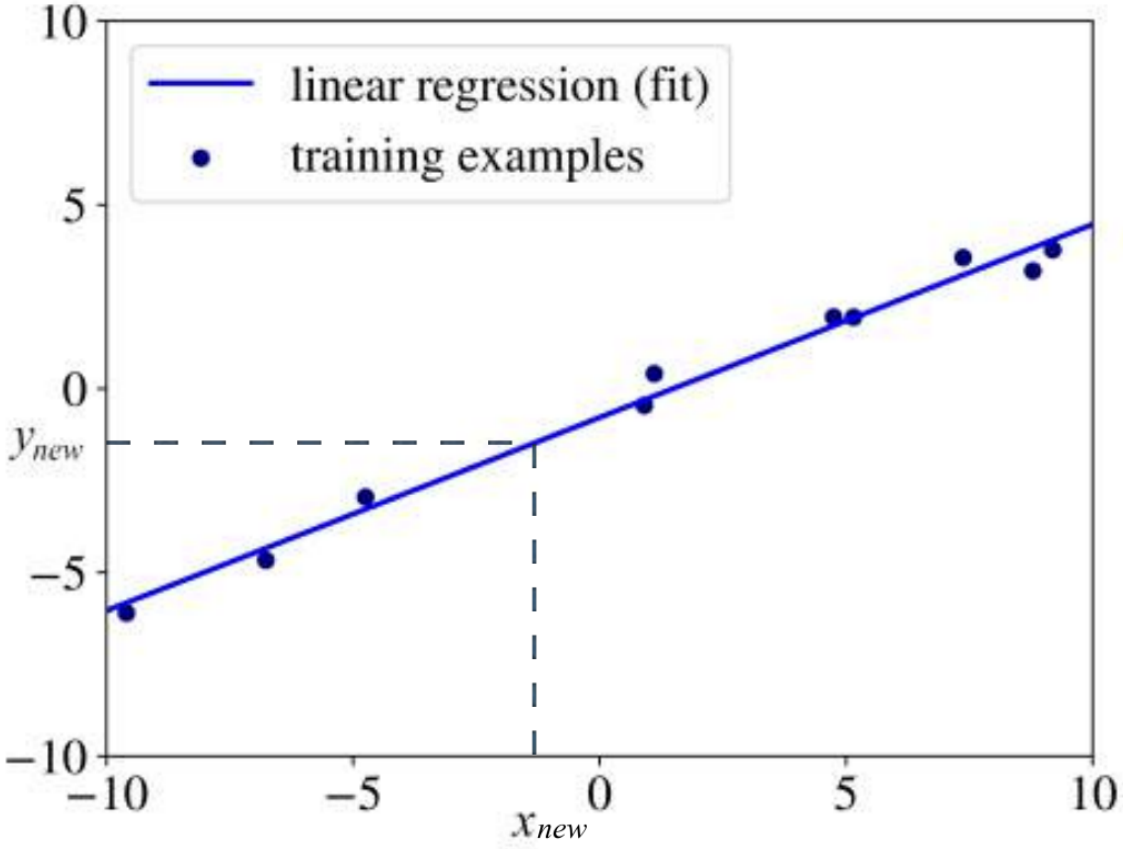


FIGURE – Régression linéaire pour des exemples unidimensionnels : la droite (bleu clair) approxime au mieux les points (bleu).

Énoncé du problème

Choix des paramètres

Objectif : trouver $(w^\star, b^\star)$ qui rendent les prédictions $\hat{y}_i = f_{w,b}(x_i)$ **aussi proches que possible** des cibles y_i.

- Interprétation géométrique :
  - $D = 1$: une **droite** d'ajustement.
  - $D = 2$: un **plan**.
  - $D > 2$: un **hyperplan**.

Question

Comment on entraine le model à trouver $(w^\star, b^\star)$?

Fonction de coût (erreur quadratique moyenne)

Cost function

- On minimise l'**erreur quadratique moyenne (MSE)** :

$$J(w, b) \;=\; \frac{1}{N}\sum_{i=1}^{N}\left(f_{w,b}(x_i) - y_i\right)^2 \;=\; \frac{1}{N}\sum_{i=1}^{N}\left(w^\top x_i + b - y_i\right)^2$$

- Objectif : $\min_{w,b} J(w, b)$

- **Perte quadratique** : pénalise davantage les grandes erreurs, **lisse** (dérivable partout) $\Rightarrow$ optimisation plus simple.

Gradient Descent — Principe

Idée générale

Gradient Descent est un algorithme d'optimisation qui ajuste les paramètres d'un modèle en suivant la direction de la pente négative du coût (loss function).

- Objectif : trouver les paramètres w, b qui minimisent la fonction de coût $J(w, b)$.
- Démarrage avec une valeur initiale aléatoire des paramètres.
- Répétition d'une mise à jour :

$$\theta \leftarrow \theta - \alpha \nabla_\theta J(\theta)$$

  où $\theta = (w, b)$, et α est le learning rate.
- Chaque étape réduit l'erreur jusqu'à atteindre (ou approcher) un minimum.

Intuition

Comme descendre une colline : à chaque pas, on avance dans la direction de la pente la plus forte, jusqu'à atteindre une vallée (minimum).

Gradient Descent Update Algorithm

Algorithme 1 : Gradient Descent Update

1: **Entrée :** jeu d'entraînement $\{(x_i, y_i)\}_{i=1}^m$, learning rate α
2: **Initialiser :** w, b
3: **repeat**
4: Calculer les prédictions : $f_{w,b}(x_i) = wx_i + b \quad \forall i$
5: Calculer les gradients :

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^{m} \big(f_{w,b}(x_i) - y_i\big) x_i$$

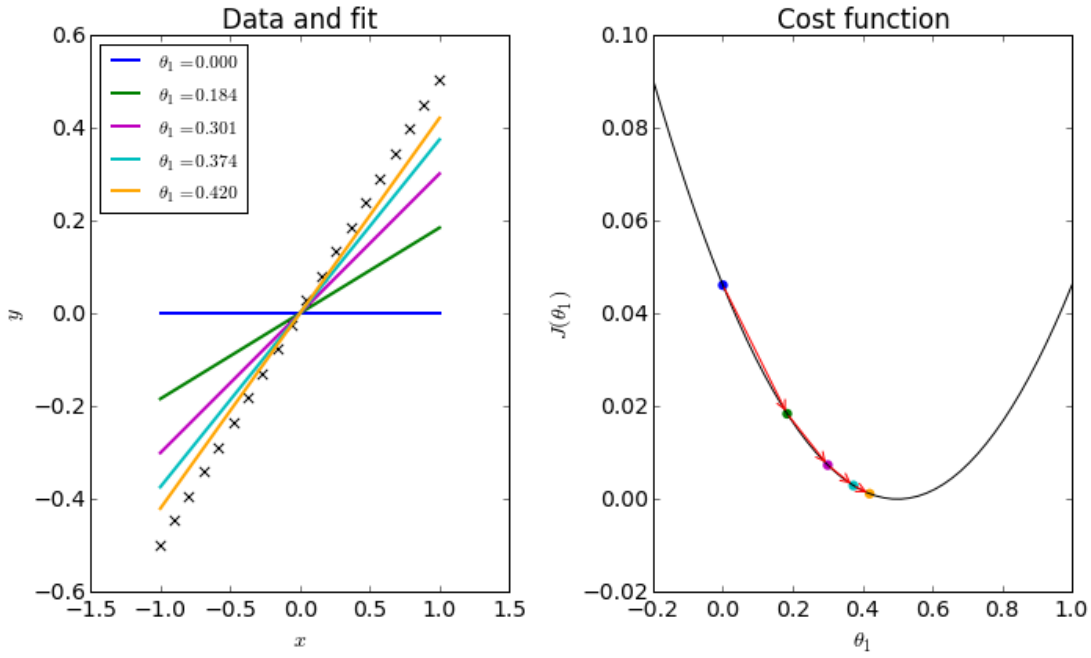
$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \big(f_{w,b}(x_i) - y_i\big)$$

6: Mettre à jour :

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}, \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

7: **until** convergence (ou nombre max d'itérations)

Global Minimum



Global Minimum

- La fonction de coût $J(w, b)$ est convexe.
- Le minimum global est le point où la fonction est minimisée.
- Gradient Descent converge vers ce point.
- Le minimum global est la solution optimale en régression linéaire.

Sur-apprentissage : exemple

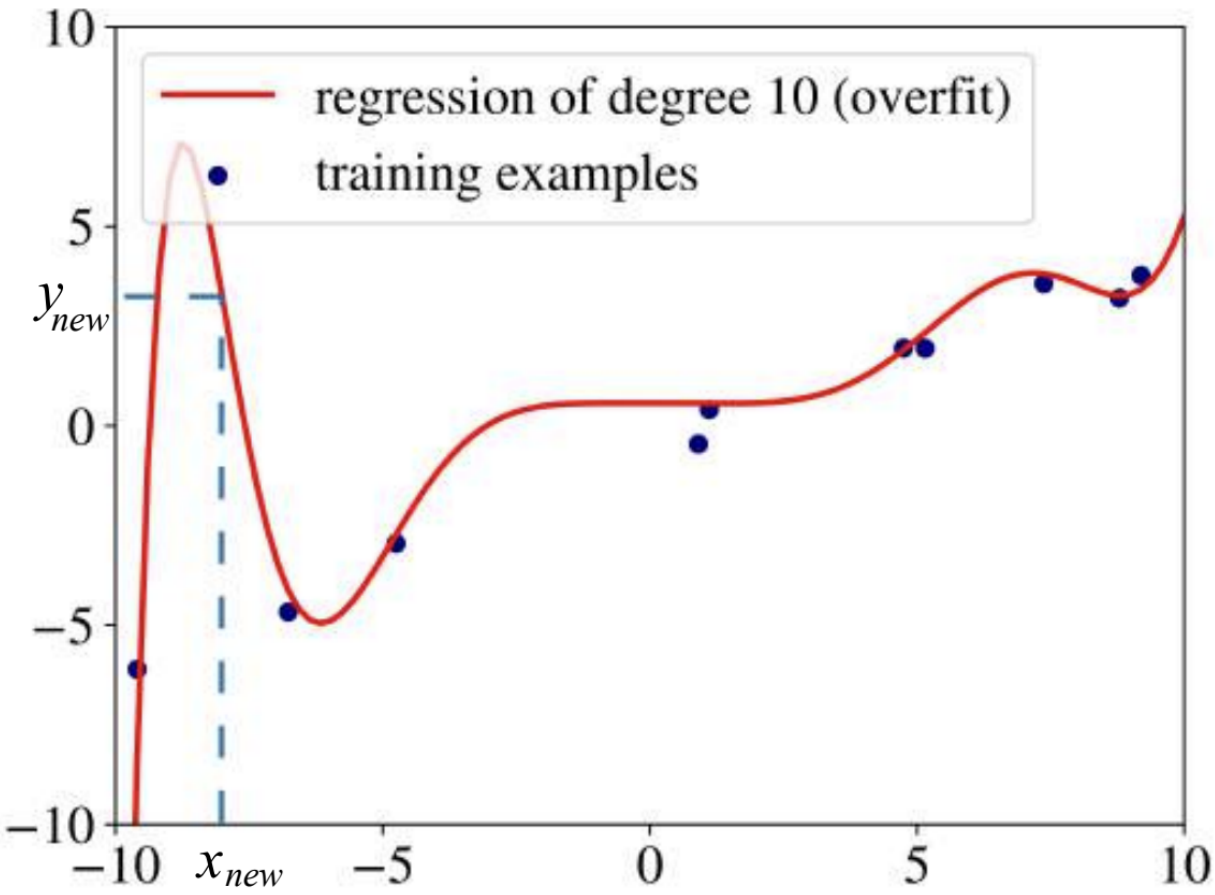


FIGURE – Overfitting : une régression polynomiale de degré élevé colle les points d'entraînement mais généralise mal (x_{new}).

À retenir

- $f_{w,b}(x) = w^\top x + b$ et $J(w, b) = \frac{1}{N} \sum (w^\top x_i + b - y_i)^2$.
- Deux voies pour optimiser : **solution fermée** (OLS) ou **descente de gradient**.
- Modèle simple $\Rightarrow$ **moins d'overfitting** que des modèles très flexibles.

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle

Classification vue d'ensemble

Définition

La classification est une tâche de **supervised learning** où l'objectif est de prédire un **label discret** (catégorie) à partir de features d'entrée.

- **Binaire** : 2 classes (ex. succès/échec, spam/non-spam).
- **Multiclasse** : plus de 2 classes (ex. reconnaître un chiffre de 0 à 9).
- **Multilabel** : plusieurs labels possibles pour un même exemple.

Exemples concrets

- Diagnostiquer une maladie (*oui*/*non*).
- Reconnaissance d'image (chat, chien, oiseau).
- Classification de texte (positif / neutre / négatif).

Classification

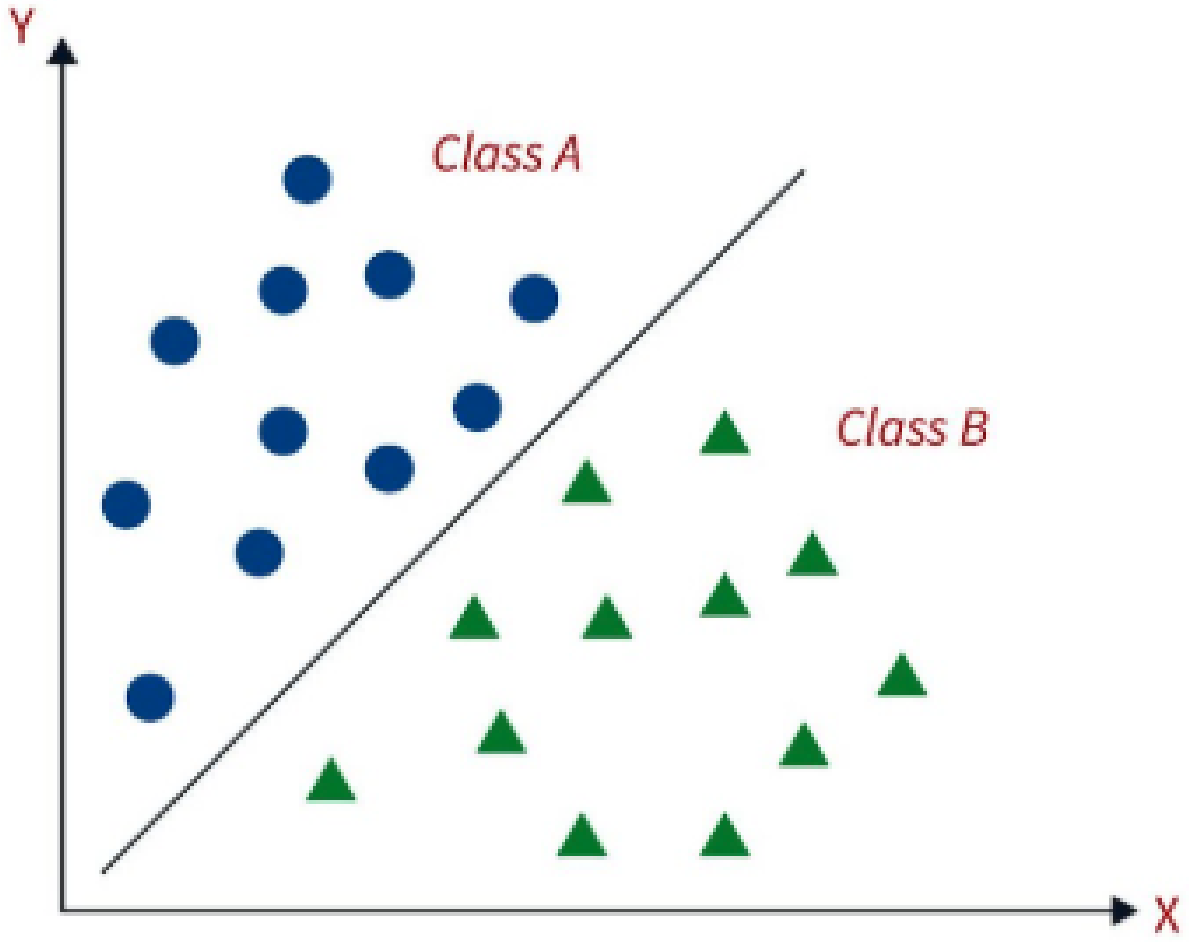


FIGURE – Classification : Séparer deux classes par une frontière de décision

Limite de la régression linéaire pour la classification

- En régression linéaire : la sortie peut être $\hat{y} \in \mathbb{R}$.
- Pour un problème de classification (succès/échec), on veut une sortie comprise entre 0 et 1.
- **Problème :** une droite peut prédire des valeurs négatives ou supérieures à 1.

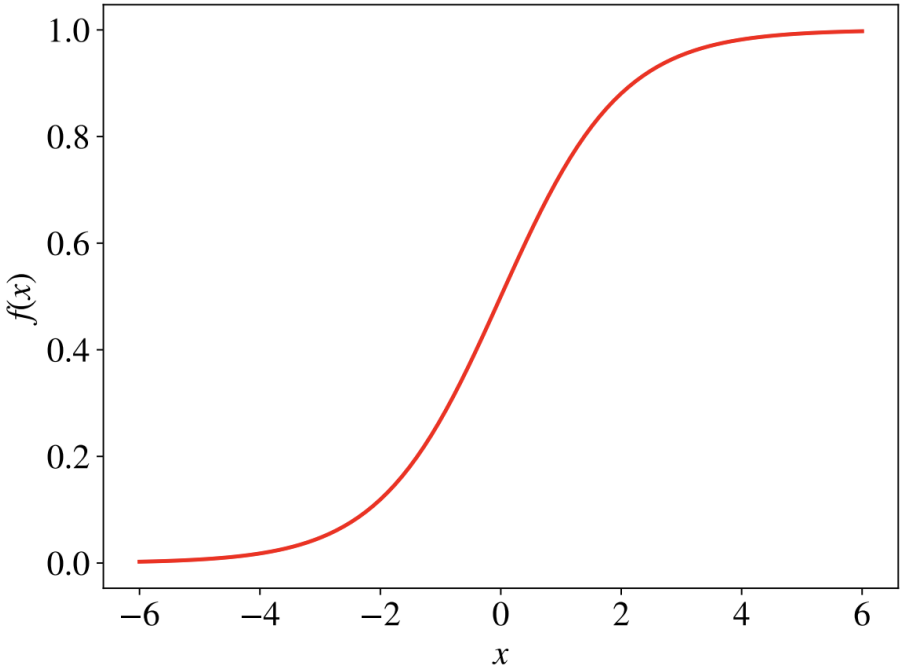
Idée

Il faut une fonction qui « écrase » les valeurs réelles dans l'intervalle $[0, 1]$.

Fonction logistique (sigmoïde)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(z)$ transforme n'importe quelle valeur réelle en un nombre entre 0 et 1.
- Interprétation probabiliste : $\sigma(z)$ = probabilité de succès (classe 1).



Régression logistique — Modèle

Logistic Function

La fonction logistique est définie tel que :

$$\hat{y} = f_{w,b}(x) \;=\; \sigma(w^\top x + b) \;=\; \frac{1}{1 + e^{-(w^\top x + b)}} \;\approx\; \mathbb{P}(y = 1 \mid x).$$

- $\hat{y}$: probabilité prédite que $y = 1$.
- $x \in \mathbb{R}^D$: vecteur des features.
- $w \in \mathbb{R}^D, b \in \mathbb{R}$: paramètres du modèle.

Exemple

Heures d'étude $x \to$ prédire la probabilité de réussir un examen. Si $\hat{y} = 0.8$, alors l'étudiant a 80% de chances de réussir.

Énoncé du problème (Logistic Regression)

Choix des paramètres

Objectif : trouver $(w^\star, b^\star)$ qui rendent les prédictions $\hat{y}_i = f_{w,b}(x_i) = \sigma(w^\top x_i + b)$ **cohérentes avec les cibles** $y_i \in \{0, 1\}$.

- **Learning Objective** : trouver $(w^\star, b^\star)$ tels que :

$$f_{w,b}(x^{(i)}) \approx 1 \quad \text{si } y^{(i)} = 1$$

$$f_{w,b}(x^{(i)}) \approx 0 \quad \text{si } y^{(i)} = 0$$

Question

Comment entraîner le modèle à trouver $(w^\star, b^\star)$?

Maximum de vraisemblance — intuition

Idée

Le Maximum de vraisemblance (MLE) consiste à choisir les paramètres du modèle qui rendent les données observées les **plus probables**.

- Données observées : 10 lancers de pièce $\rightarrow$ 7 faces, 3 piles.
- Soit $p =$ probabilité d'obtenir face.
- Exemple : pour une séquence (F,F,F,F,F,F,F,P,P,P), la probabilité est

$$p \times p \times p \times p \times p \times p \times p \times (1-p) \times (1-p) \times (1-p).$$

- En simplifiant, cela donne

$$p^7(1-p)^3.$$

Maximum de vraisemblance — intuition

fonction de vraisemblance

Comme il existe plusieurs façons d'obtenir 7 faces parmi 10 :

$$L(p) = \binom{10}{7} p^7 (1-p)^3$$

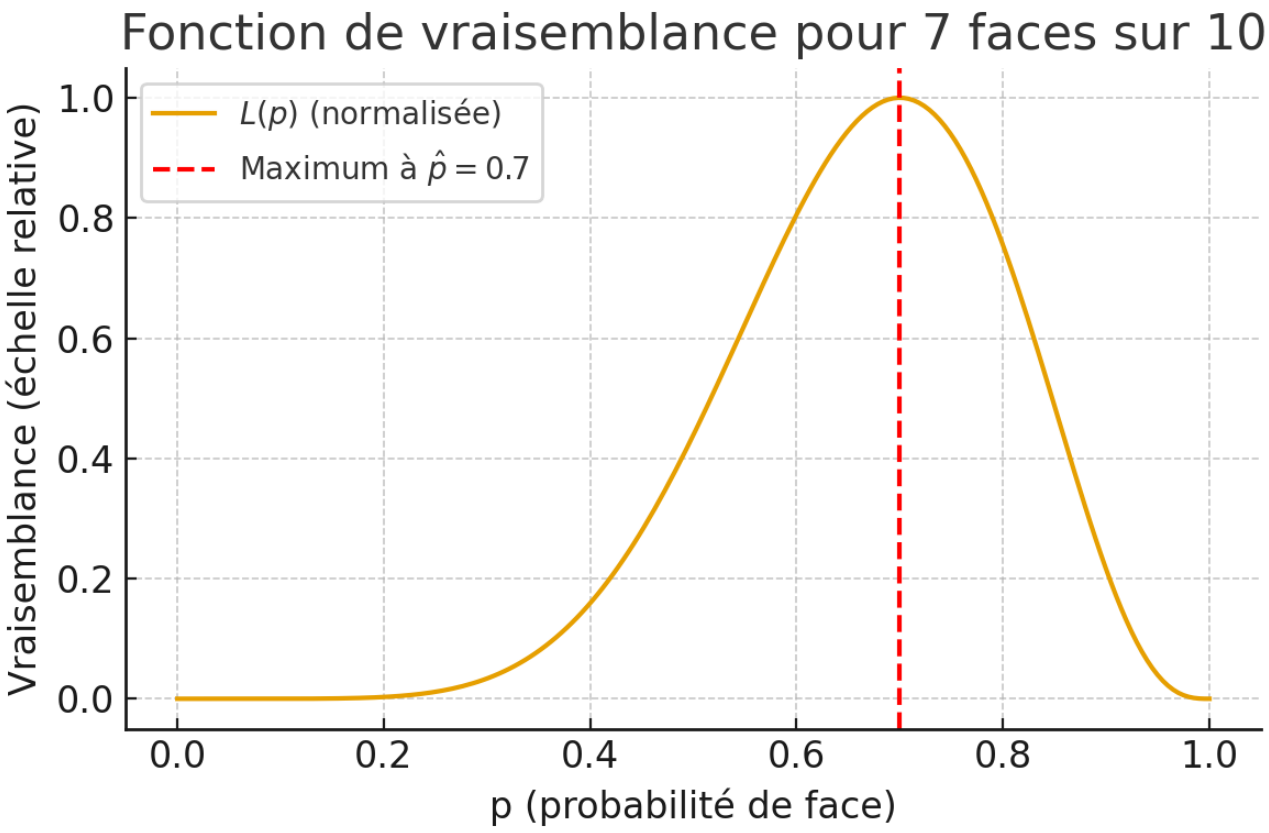
Explication

Cette fonction mesure : « Si p valait telle valeur, à quel point mes données (7F, 3P) seraient-elles probables ? »

Maximum de vraisemblance — exemple

- Le **MLE** est la valeur de p qui rend $L(p)$ la plus grande possible.
- Ici, $L(p)$ est maximale pour

$$\hat{p} = \frac{7}{10} = 0.7$$



Vraisemblance en régression logistique

- En classification binaire : $y_i \in \{0, 1\}$.
- Modèle : $f_{w,b}(x_i) = P(y_i = 1 \mid x_i)$.
- La probabilité d'un point est :

$$P(y_i \mid x_i; w, b) = [f_{w,b}(x_i)]^{y_i}[1 - f_{w,b}(x_i)]^{1-y_i}.$$

- Vraisemblance du dataset :

$$\mathcal{L}(w, b) = \prod_{i=1}^{N}[f_{w,b}(x_i)]^{y_i}[1 - f_{w,b}(x_i)]^{1-y_i}.$$

Log-vraisemblance

- Problème : le produit est numériquement instable (valeurs très petites).
- Solution : on prend le logarithme :

$$\ell(w, b) = \sum_{i=1}^{N} \Big[y_i \log f_{w,b}(x_i) + (1 - y_i) \log(1 - f_{w,b}(x_i)) \Big].$$

- Plus simple à manipuler et à dériver.

Exemple

Si on observe $y = [1, 0, 1]$, et $f_{w,b}(x) = [0.9, 0.2, 0.7]$:

$$\ell = \log(0.9) + \log(1 - 0.2) + \log(0.7).$$

MLE et fonction de coût

- Objectif :

$$\max_{w,b} \ \ell(w, b)$$

- Équivalent à :

$$\min_{w,b} \ J(w, b) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log f_{w,b}(x_i) + (1 - y_i) \log(1 - f_{w,b}(x_i)) \Big]$$

- Cette fonction $J(w, b)$ est appelée :
  - **log-loss** ou entropie croisée
  - Fonction de coût standard de la régression logistique

Conclusion

En régression logistique, entraîner le modèle par MLE revient à minimiser l'entropie croisée.

Décision et frontière

- Règle de décision (seuil τ) :

$$\hat{y} = \begin{cases} 1, & \text{si } f_{w,b}(x) \geq \tau \\ 0, & \text{sinon} \end{cases}$$

- Frontière linéaire : $w^\top x + b = 0$.
- Plus le seuil est ajusté, plus on peut contrôler la sensibilité/spécificité.

Exemple

Si un étudiant a étudié 5h : $\hat{y} = 0.72 \rightarrow$ prédiction « succès », avec $\tau = 0.5$.

Batch Gradient Descent (BGD)

Algorithme 2 : Batch Gradient Descent

1: **Entrée :** jeu d'entraînement $\{(x_i, y_i)\}_{i=1}^m$, learning rate α
2: **Initialiser :** $w \in \mathbb{R}^n,\ b \in \mathbb{R}$
3: **repeat**
4: Calculer les prédictions : $f_{w,b}(x_i) = w^\top x_i + b \quad \forall i$
5: Calculer les gradients :

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \big(f_{w,b}(x_i) - y_i\big) x_{i,j}, \quad \forall j \in \{1, \dots, n\}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \big(f_{w,b}(x_i) - y_i\big)$$

6: Mettre à jour :

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}, \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

7: **until** convergence (ou nombre max d'itérations)

À retenir

- Sortie $\in (0, 1)$ via $\sigma(\cdot) \Rightarrow$ probabilités & seuils de décision ajustables.
- Optimisation par **maximum de vraisemblance** $\Leftrightarrow$ minimisation **entropie croisée**.
- Pas de solution algébrique fermée $\Rightarrow$ **descente de gradient** (plein-batch, mini-batch, SGD).
- Problème **convexe** en $w, b \Rightarrow$ optimum **global**.
- Frontière **linéaire**.

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle

Lecture de la structure d'un arbre de décision

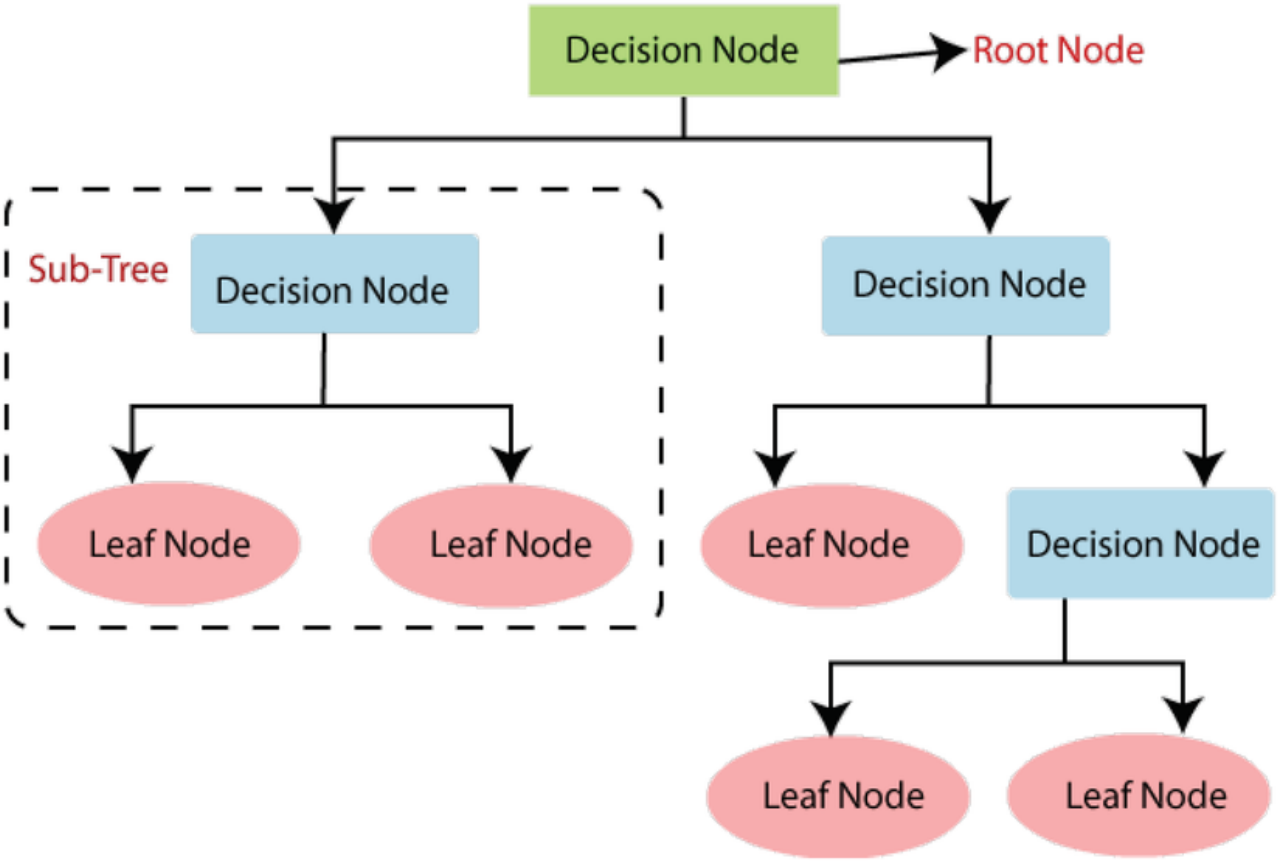


FIGURE – Structure d'un arbre décisionnel

Idée générale

- Apprendre des règles **SI ... ALORS ...** en découpant récursivement l'espace des features.
- Chaque nœud teste une condition simple (seuil numérique ou appartenance à une catégorie).
- Les feuilles prédisent une **classe** (ou une distribution de classes).
- Objectif : obtenir des sous-ensembles **purs** (une classe dominante) avec des règles **interprétables**.

Problème

- Données supervisées : $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, $x_i \in \mathbb{R}^D$, $y_i \in \{1, \ldots, C\}$ (binaire ou multiclasse).
- Construire un arbre qui **partitionne** $\mathcal{D}$ en sous-ensembles de plus en plus **homogènes** en y.
- À chaque nœud : choisir le **meilleur split** (feature & seuil) qui **réduit l'impureté**.

Mesures d'impureté (classification)

- **Gini** : $G(S) = 1 - \sum_{c=1}^{C} p_c^2$, où p_c est la proportion de la classe c dans l'ensemble S.
- **Entropie** : $H(S) = -\sum_{c=1}^{C} p_c \log_2 p_c$.
- **Gain** d'un split $S \to (S_L, S_R)$:

$$\text{Gain} = \text{Impureté}(S) - \frac{|S_L|}{|S|}\text{Imp}(S_L) - \frac{|S_R|}{|S|}\text{Imp}(S_R).$$

- Choisir le split qui **maximise** le gain (ou **minimise** l'impureté pondérée).

Algorithme (CART/ID3, top-down glouton)

1. **Si** le nœud est **pur** ou critère d'arrêt atteint $\Rightarrow$ créer une **feuille** (classe majoritaire).
2. Sinon, pour chaque feature (et seuils candidats) :
   - Calculer l'**impureté** des sous-ensembles après split.
   - Évaluer le **gain** (Gini ou entropie).
3. Choisir le split avec **gain maximal**.
4. **Répartir** les exemples à gauche/droite, puis **répéter** récursivement.

Jeu de données jouet (12 emails) — table

ID	Longueur	contient_FREE ?	nb_liens	Label
1	130	Oui	4	Spam
2	150	Non	5	Spam
3	110	Oui	3	Spam
4	140	Non	2	Spam
5	95	Oui	3	Spam
6	80	Oui	4	Spam
7	90	Oui	2	Non_spam
8	70	Non	2	Spam
9	85	Non	5	Non_spam
10	60	Non	1	Non_spam
11	50	Non	0	Non_spam
12	100	Non	2	Non_spam

Exemple complet pas à pas

- Données binaire **Spam** (S) / **Not** (N), N=12 : 7 S, 5 N.
- Impureté à la racine (Gini) :

$$G(S) = 1 - \left(\frac{7}{12}\right)^2 - \left(\frac{5}{12}\right)^2 = 1 - \frac{49+25}{144} = \frac{70}{144} \approx 0.486.$$

- Features candidates : `longueur` $>$ 100? (numérique), `contient_FREE?` (bool), `nb_liens` $>$ 2? (numérique).

Étape 1 — choix du *meilleur* split (à la racine)

Split A : `longueur` > 100?

S_L (OUI) : 4 ex. $\rightarrow$ 4S/0N $\Rightarrow G = 0$. S_R (NON) : 8 ex. $\rightarrow$ 3S/5N
$\Rightarrow G = 1 - \left(\frac{3}{8}\right)^2 - \left(\frac{5}{8}\right)^2 = 0.469$.
$\Rightarrow$ Gini pondéré $= \frac{4}{12} \cdot 0 + \frac{8}{12} \cdot 0.469 = 0.312$. **Gain** $= 0.486 - 0.312 = 0.174$.

Split B : `contient_FREE`?

S_L (VRAI) : 5 ex. $\rightarrow$ 4S/1N $\Rightarrow G = 0.32$. S_R (FAUX) : 7 ex. $\rightarrow$ 3S/4N
$\Rightarrow G \approx 0.490$.
Gini pondéré $\approx \frac{5}{12} \cdot 0.32 + \frac{7}{12} \cdot 0.490 \approx 0.419$. **Gain** ≈ 0.067.

Split C : `nb_liens` > 2?

Deux sous-ensembles de 6 : 5S/1N $\Rightarrow G \approx 0.278$ et 2S/4N $\Rightarrow G \approx 0.444$.
Gini pondéré $= 0.5(0.278) + 0.5(0.444) = 0.361$. **Gain** ≈ 0.125.

$\Rightarrow$ **Meilleur split** : `longueur` > 100?.

Étape 2 — poursuivre côté non pur

- Après `longueur` > 100? :

$$\underbrace{\text{Gauche (OUI)}}_{\text{4S/0N, feuille pure}} \qquad \underbrace{\text{Droite (NON)}}_{\text{8 ex. : 3S/5N, non pur}}$$

Sur le nœud droit (8 ex.) : tester `contient_FREE`?

S_L (VRAI) : 3 ex. $\rightarrow$ 2S/1N $\Rightarrow G = 1 - (\frac{2}{3})^2 - (\frac{1}{3})^2 = 0.444$.
S_R (FAUX) : 5 ex. $\rightarrow$ 1S/4N $\Rightarrow G = 1 - (\frac{1}{5})^2 - (\frac{4}{5})^2 = 0.32$.
Gini pondéré $= \frac{3}{8} \cdot 0.444 + \frac{5}{8} \cdot 0.32 \approx 0.367$. **Gain** $= 0.469 - 0.367 \approx 0.102$.

(Moins bon) tester `nb_liens` > 2?

Deux sous-ensembles 4/4 : 2S/2N $\Rightarrow G = 0.5$ et 1S/3N $\Rightarrow G = 0.375$.
Gini pondéré $= 0.4375$. **Gain** $= 0.469 - 0.438 \approx 0.031$.

$\Rightarrow$ On **splitte** par `contient_FREE`?.

Feuilles, vote majoritaire & proba

- **Feuille gauche** (après `longueur` > 100?) : 4S/0N $\Rightarrow$ prédiction **Spam**
- **Feuille droite-gauche** (`FREE=VRAI`) : 2S/1N $\Rightarrow \hat{y} =$ **Spam**
- **Feuille droite-droite** (`FREE=FAUX`) : 1S/4N $\Rightarrow \hat{y} =$ **Not**

Chemin de décision — prédire un nouvel email

Ex. 1

`longueur` $= 120 \Rightarrow$ `longueur` $> 100? =$ OUI $\Rightarrow$ **feuille 4S/0N**.
Prédiction : **Spam**.

Ex. 2

`longueur` $= 90$ (NON), `FREE=FAUX` $\Rightarrow$ feuille **1S/4N**.
Prédiction : **Not**.

Arbre entraîné (vue globale)

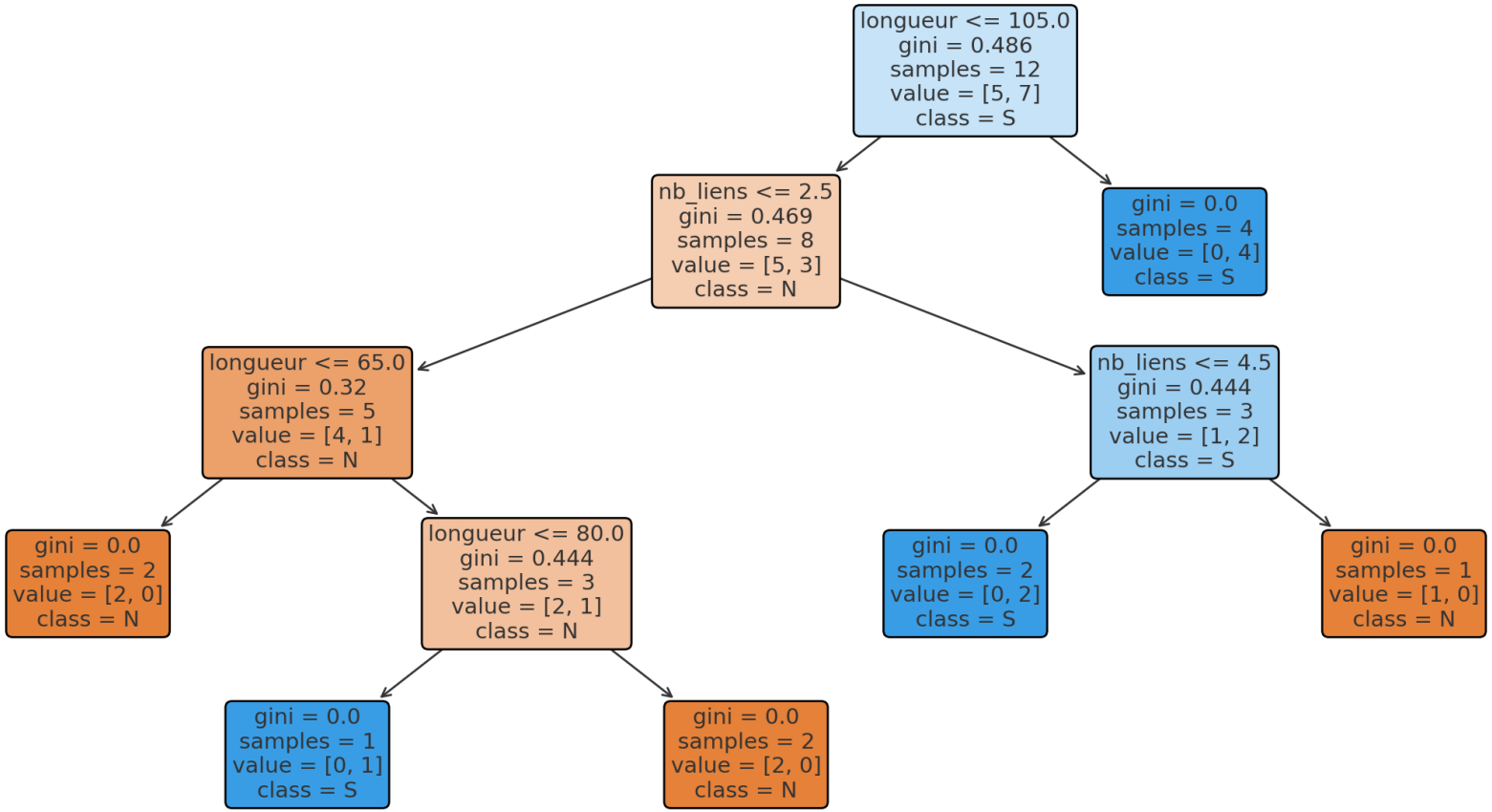


FIGURE – Arbre obtenu sur le jeu exemple : règles, effectifs, impuretés, classe majoritaire.

Arrêts & élagage (pruning)

- **Pré-élagage** (arrêt anticipé) : profondeur max, `min_samples_split`, `min_samples_leaf`, gain min.
- **Post-élagage** (coût-complexité) :

$$R_\alpha(T) = R(T) + \alpha\,|\text{feuilles}(T)|, \quad \alpha \geq 0$$

Sélection de α par validation croisée $\Rightarrow$ compromis **biais/variance**.

Visualisation 2D (exemple)

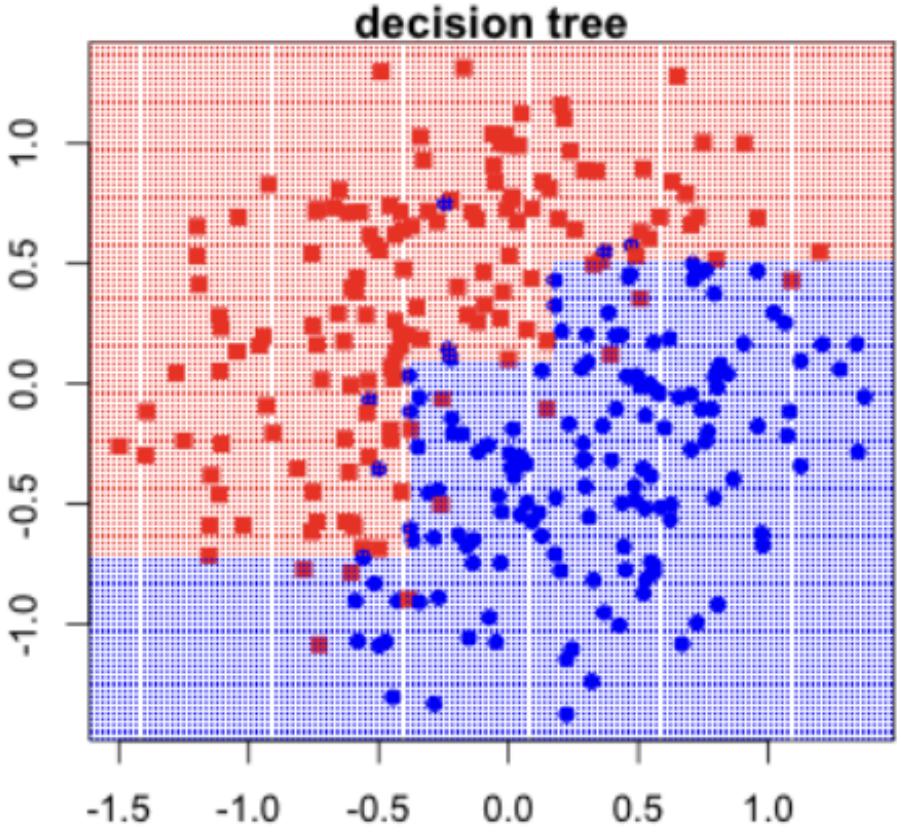


FIGURE – Régions de décision par coupes **alignées sur les axes**.

À retenir

- Un arbre classe via des **règles** apprises pour **maximiser** la **baisse d'impureté**.
- **Gini** et **Entropie** mesurent l'homogénéité des nœuds.
- **Élagage** indispensable pour contrôler la complexité et généraliser.
- Base des **forêts aléatoires** et **boosting**, très performants en pratique.

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle

Exemple SVM 2D : frontière & marge

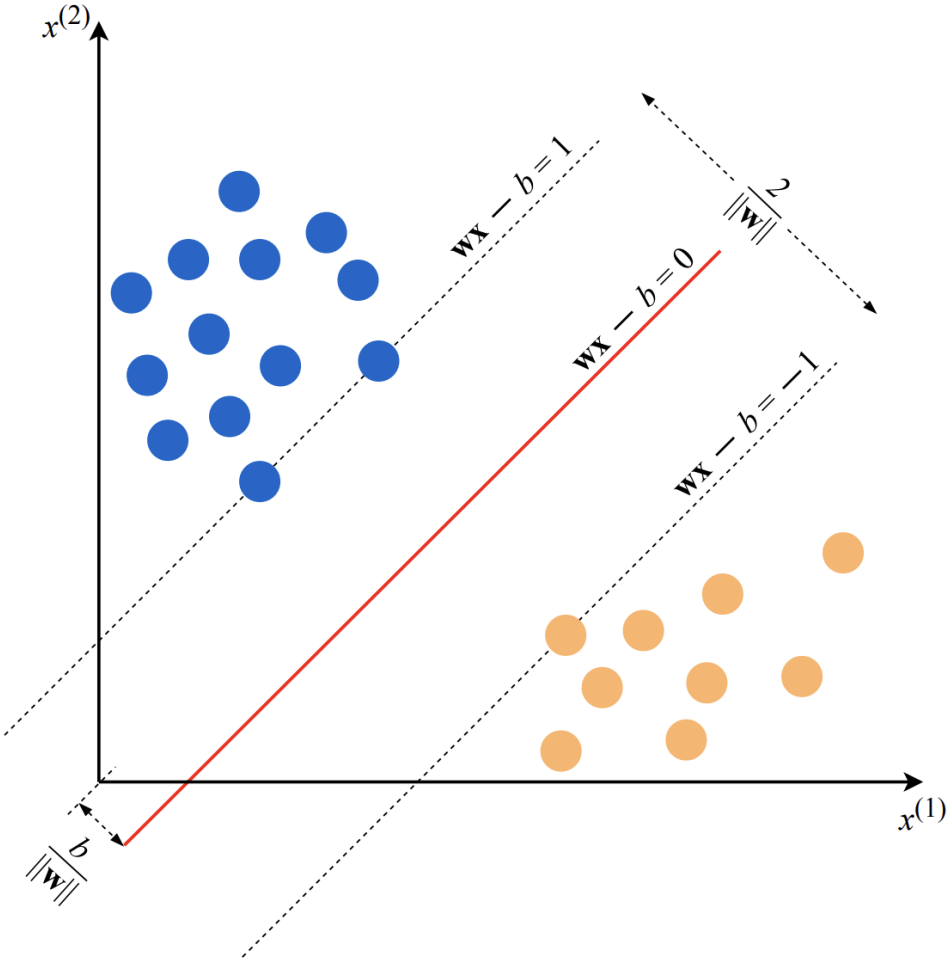


FIGURE – Exemple d'un modèle SVM en deux dimensions : hyperplan $w^\top x - b = 0$ et marges $w^\top x - b = \pm 1$.

SVM linéaire : modèle & frontière de décision

- Le SVM cherche un **hyperplan** qui sépare les classes :

$$w^\top x - b = 0$$

- La **prédiction** pour une entrée x :

$$f(x) = \mathrm{sign}\big(w^\top x - b\big) \in \{-1, +1\}$$

- La **frontière de décision** est l'ensemble des x tels que $w^\top x - b = 0$.

Entraînement SVM : marge & optimisation

- Contraintes de séparation (cas linéairement séparable) :

$$y_i(w^\top x_i - b) \geq 1 \quad \text{pour } i = 1, \ldots, N$$

- **Marge** géométrique : distance entre les hyperplans $w^\top x - b = 1$ et $w^\top x - b = -1$:

$$\text{marge} = \frac{2}{\|w\|}$$

- Problème d'optimisation (SVM dur) :

$$\min_{w,b} \|w\| \quad \text{s.c.} \quad y_i(w^\top x_i - b) \geq 1$$

- Intuition : **maximiser la marge** $\Rightarrow$ meilleure **généralisation**.

Pourquoi une **marge souple** ?

- Dans la vraie vie, les données ne sont pas toujours **linéairement séparables**.
- On autorise donc quelques **violations** de la marge (voire des erreurs) mais on les **pénalise**.
- Idée clé : **équilibrer** deux objectifs
  1. **Large marge** (bonne généralisation),
  2. **Peu d'erreurs** sur les données d'entraînement.

Illustration : marge souple et violations

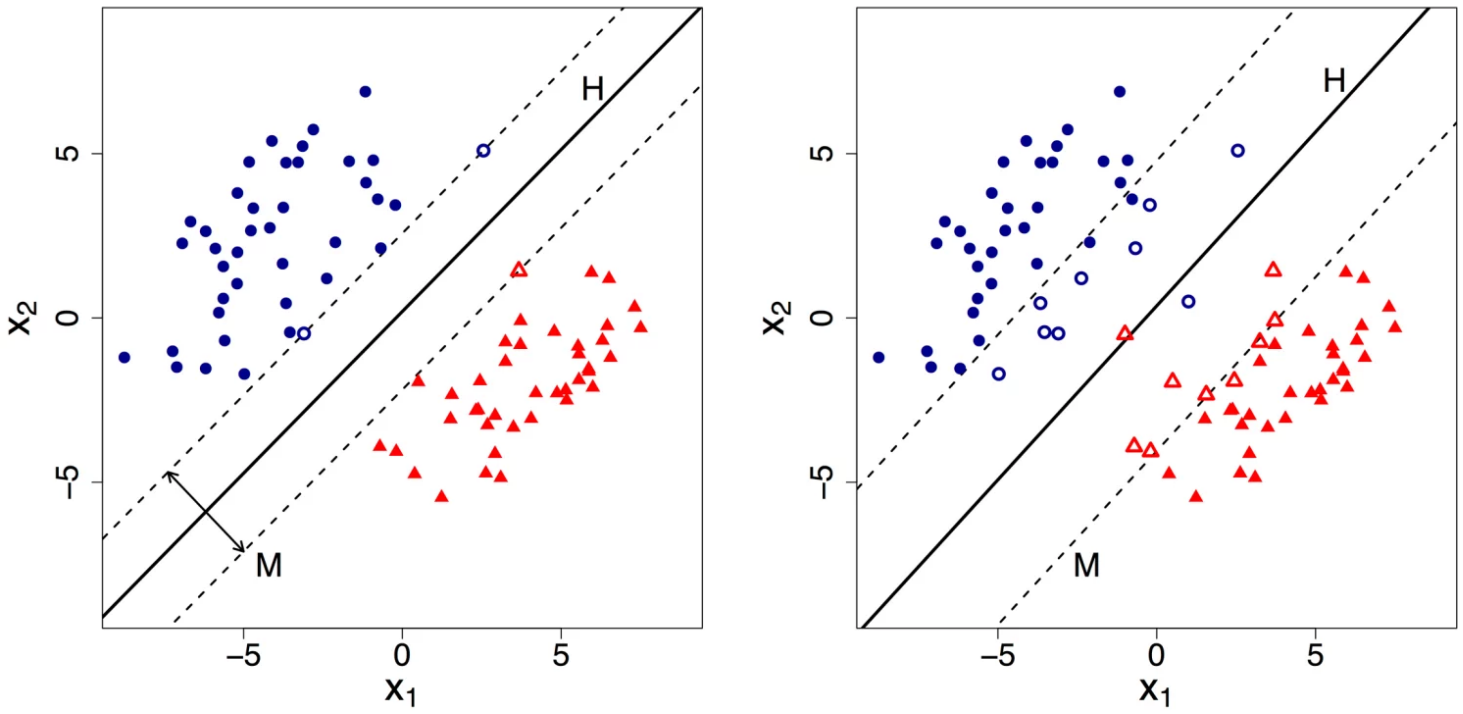
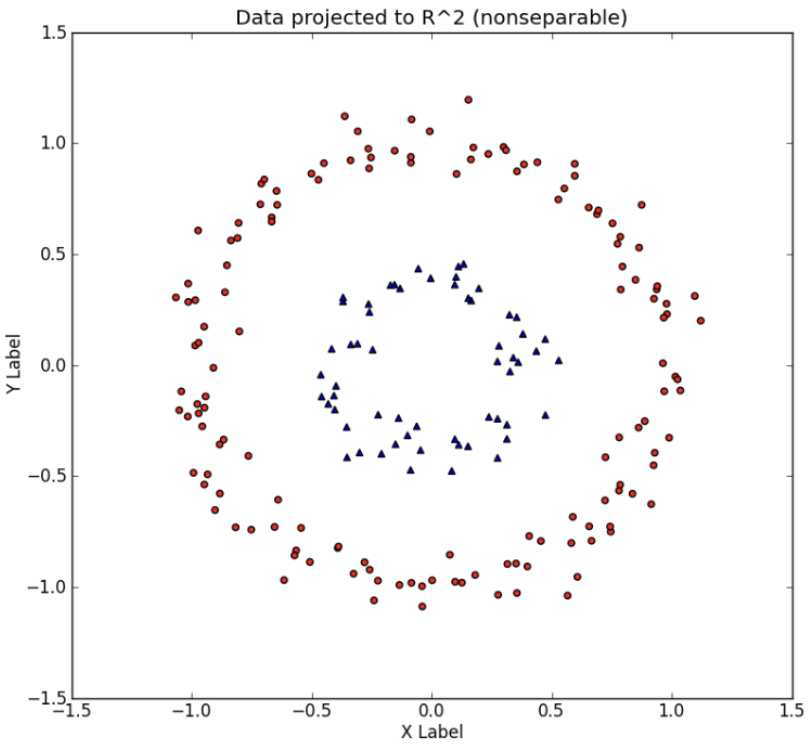


FIGURE – À gauche : Hard Margin SVM. À droite : Soft Margin SVM.

Intuition : changer d'espace pour séparer

- Parfois, aucune **droite/hyperplan** ne sépare les classes dans l'espace d'origine.
- Idée : **projeter** les données dans un espace de **plus grande dimension** où la séparation devient **linéaire**.
- Le **kernel trick** permet de **travailler comme si** on avait fait cette projection, **sans** la calculer explicitement.

Exemple visuel : 2D non séparable → 3D séparable



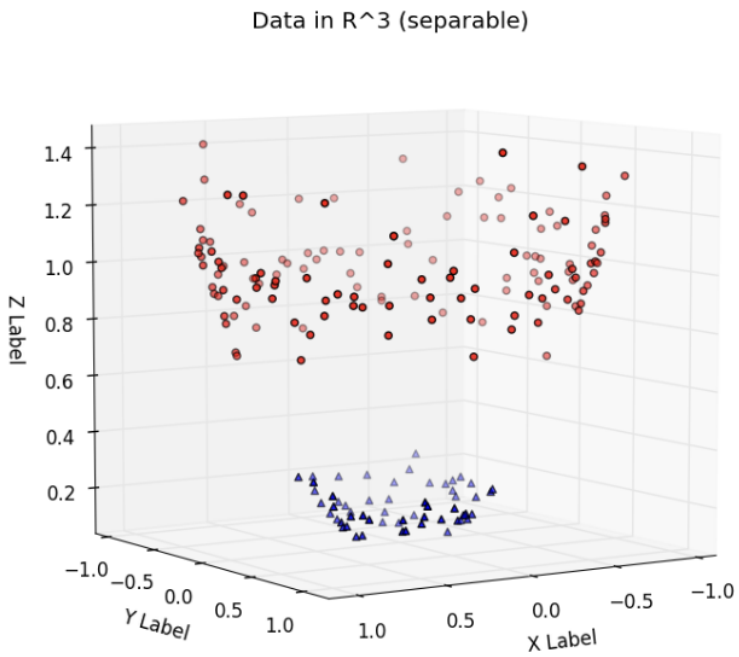


FIGURE – À gauche : données non séparables en 2D. À droite : après transformation (ex. RBF), séparation linéaire en 3D.

Qu'est-ce qu'un **noyau** ?

- Un **noyau** $k(x, x')$ mesure une **similarité** qui correspond au **produit scalaire** dans l'espace transformé.
- On manipule directement $k(x, x')$ **sans** construire les features de haute dimension.
- Exemples courants :
  - **Linéaire** : $k(x, x') = x^\top x'$.
  - **Polynomiale (degré d)** : $k(x, x') = (x^\top x' + c)^d$ (le cas $d = 2$ correspond à « quadratique »).
  - **RBF/Gaussien** : $k(x, x') = \exp\big(-\gamma \|x - x'\|^2\big)$.

À retenir

- **Marge souple** = autoriser des erreurs mais **pénaliser** via la perte hinge.
- C contrôle le compromis **marge large** vs **peu d'erreurs**.
- **Kernel trick** : gérer la **non-linéarité** sans construire explicitement des features de haute dimension.
- **RBF** est un choix robuste ; ajuster C **et** γ par validation croisée.

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux
3.1 Régression linéaire
3.2 Régression logistique
3.3 Arbres de décision (classification)
3.4 SVM
3.5 k-Nearest Neighbors (kNN)

4. Évaluation et sélection de modèle

Illustration : k-Nearest Neighbors

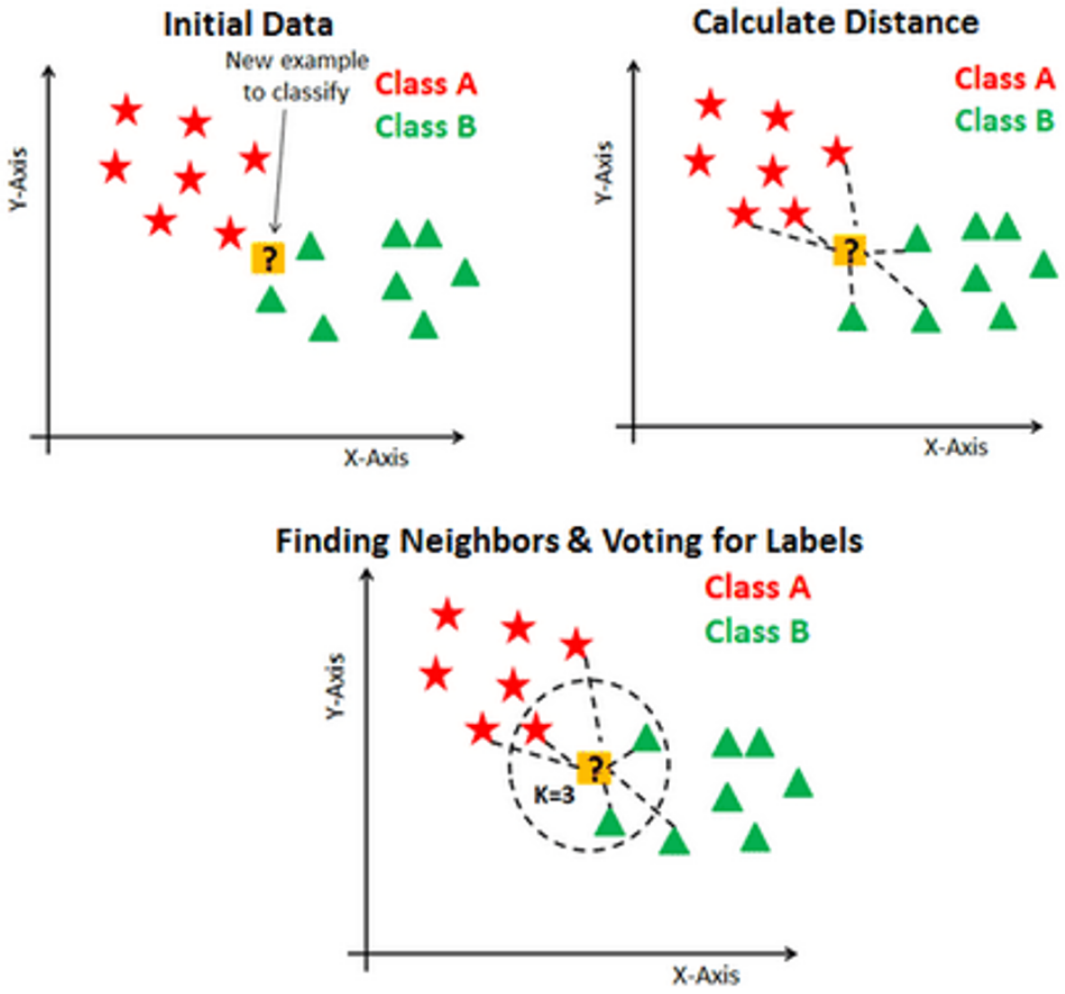


FIGURE – Fonctionnement du k-Nearest Neighbors

kNN : intuition

- **Apprentissage non paramétrique** et **paresseux** : on *garde* tous les exemples d'entraînement.
- Pour un nouvel exemple x, on cherche ses k **plus proches voisins** dans le train.
- **Classification** : vote majoritaire des k voisins.
- **Régression** : moyenne (ou moyenne pondérée) des labels des k voisins.
- Deux choix clés (hyperparamètres) : k et la **métrique de distance**.

Distances et similarités

- **Euclidienne** : $d(x, z) = \sqrt{\sum_{j=1}^{D} (x^{(j)} - z^{(j)})^2}$.
- **Cosinus** (similarité) :

$$s(x, z) = \frac{\sum_{j=1}^{D} x^{(j)} z^{(j)}}{\|x\|_2 \, \|z\|_2} \quad \Rightarrow \quad d_{\cos}(x, z) = 1 - s(x, z).$$

- Autres : **Chebyshev**, **Mahalanobis**, **Hamming** (binaire).
- **Normalisation** importante : euclidienne $\Rightarrow$ mettre les features sur la même échelle ; cosinus $\Rightarrow$ normaliser $x \leftarrow x/\|x\|_2$.

kNN : procédure

1. **Choisir** k et la **métrique**.
2. (Optionnel) **Standardiser** / **normaliser** les features.
3. Pour chaque requête x :
   1. Calculer la distance de x à tous les exemples d'entraînement.
   2. Garder les k **plus proches** $R_k(x)$.
   3. **Prédire** :
      - **Classification (vote)** : $\hat{y} = \arg\max_c \sum_{i \in R_k(x)} \mathbf{1}[y_i = c]$.
      - **Pondéré** : poids $w_i = \frac{1}{d(x, x_i) + \varepsilon}$.
      - **Régression** : $\hat{y} = \frac{\sum_{i \in R_k(x)} w_i y_i}{\sum_{i \in R_k(x)} w_i}$ (avec $w_i = 1$ si non pondéré).

Impact de k

- k **petit** : frontière *très locale* $\Rightarrow$ **variance** élevée, sensible au bruit.
- k **grand** : lisse la décision $\Rightarrow$ **biais** plus fort.
- Choix pratique : **validation croisée** (grid search) ; k impair en binaire pour limiter les égalités.
- Toujours vérifier la **courbe biais–variance** (erreur train/val).

Coûts et accélérations

- **Entraînement** : quasi nul (on mémorise les données).
- **Prédiction** naïve : $O(ND)$ par requête (distances à N points de dimension D).
- **Accélérations** : structures d'index (k-d tree, ball tree) ; recherche **approchée** (HNSW, etc.) pour grands N.
- **Curse de la dimension** : en grande dimension, les distances « se ressemblent » $\Rightarrow$ penser **sélection de variables** et **réduction dimensionnelle** (ex. PCA).

À retenir

- kNN : modèle **simple**, **efficace** en faible dimension et quand la notion de proximité **fait sens**.
- Hyperparamètres majeurs : k et la **métrique de distance** (+ normalisation).
- Coût d'inférence **élevé** si N grand $\Rightarrow$ penser indexation/approximation.
- Interprétation locale possible via une **vision linéaire régularisée** autour de la requête.

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle
4.1 Métriques principales
4.2 Biais/variance

Plan

1. Introduction

2. Les données : le cœur du ML

3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle
4.1 Métriques principales
4.2 Biais/variance

Métriques principales — Vue d'ensemble

- Une métrique **quantifie la performance** d'un modèle sur des données de test.
- Le choix dépend du **problème** (binaire, multi-classe, déséquilibre...).
- Exemples de métriques en classification :
  - **Accuracy**
  - **Precision**, **Recall**, **F1**
  - **ROC**, **AUC**

Métriques principales — Tableau comparatif

Métrique	Utilisation	Limite
Accuracy	Vue globale des performances	Trompeur si déséquilibre
Precision	Erreur sur positifs prédits	Peut ignorer des vrais positifs
Recall	Capacité à détecter les positifs	Peut générer trop de FP
F1-score	Compromis P/R	Pas toujours interprétable
ROC-AUC	Comparaison globale de modèles	Moins intuitif pour données déséquilibrées

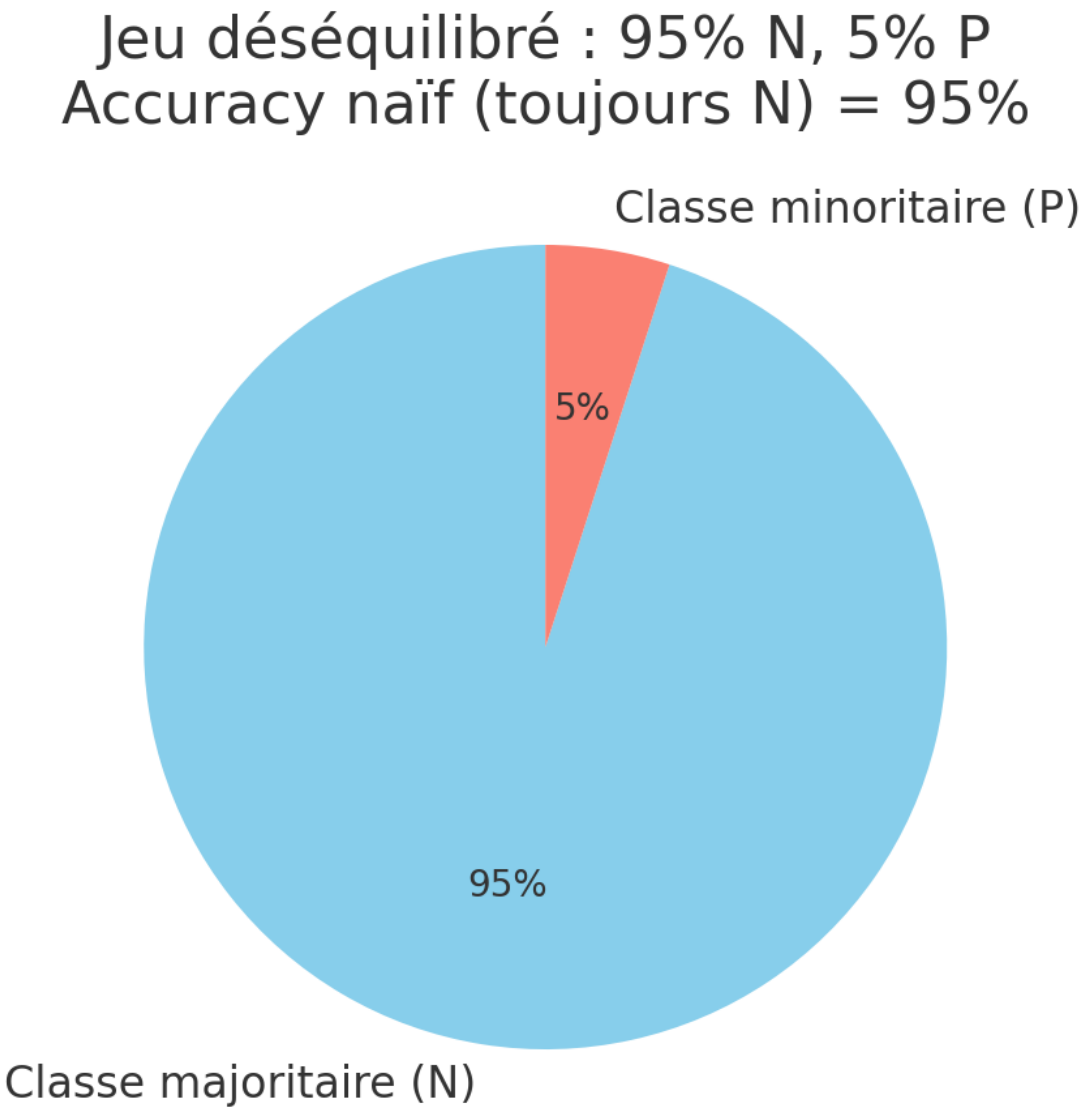
Accuracy

- Définition :

$$\text{Accuracy} = \frac{\#\text{prédictions correctes}}{\#\text{total d'exemples}}$$

- Intuition : probabilité qu'une prédiction soit correcte.
- **Limite** : peut être trompeur si les classes sont déséquilibrées.

Accuracy et classes déséquilibrées



Précision, Rappel et F1-score

- **Précision (Precision)** : parmi les positifs prédits, combien sont corrects ?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Rappel (Recall)** : parmi les vrais positifs, combien détectés ?

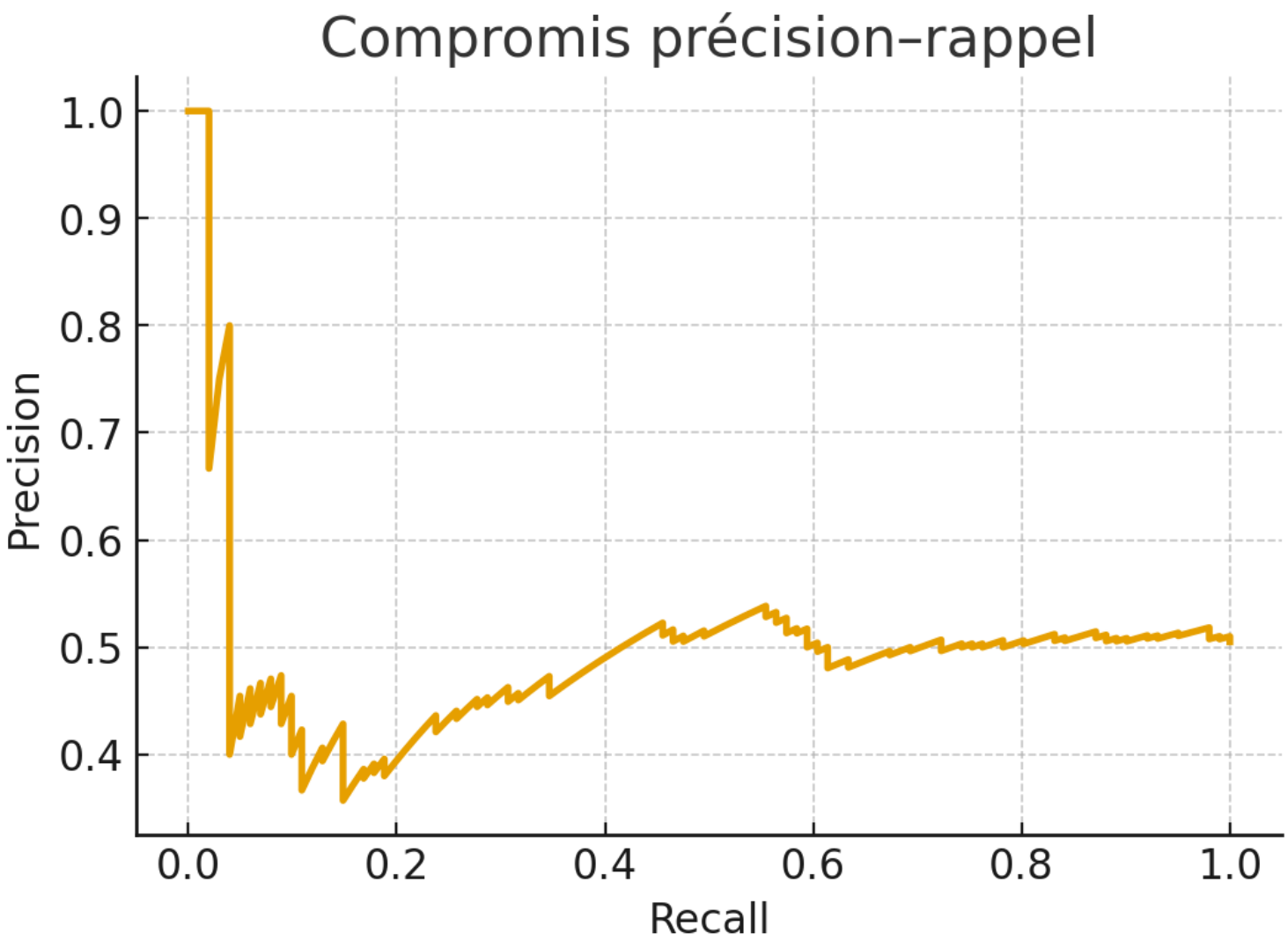
$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score** : moyenne harmonique

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Trade-off** : augmenter l'un fait souvent baisser l'autre.

Compromis précision–rappel



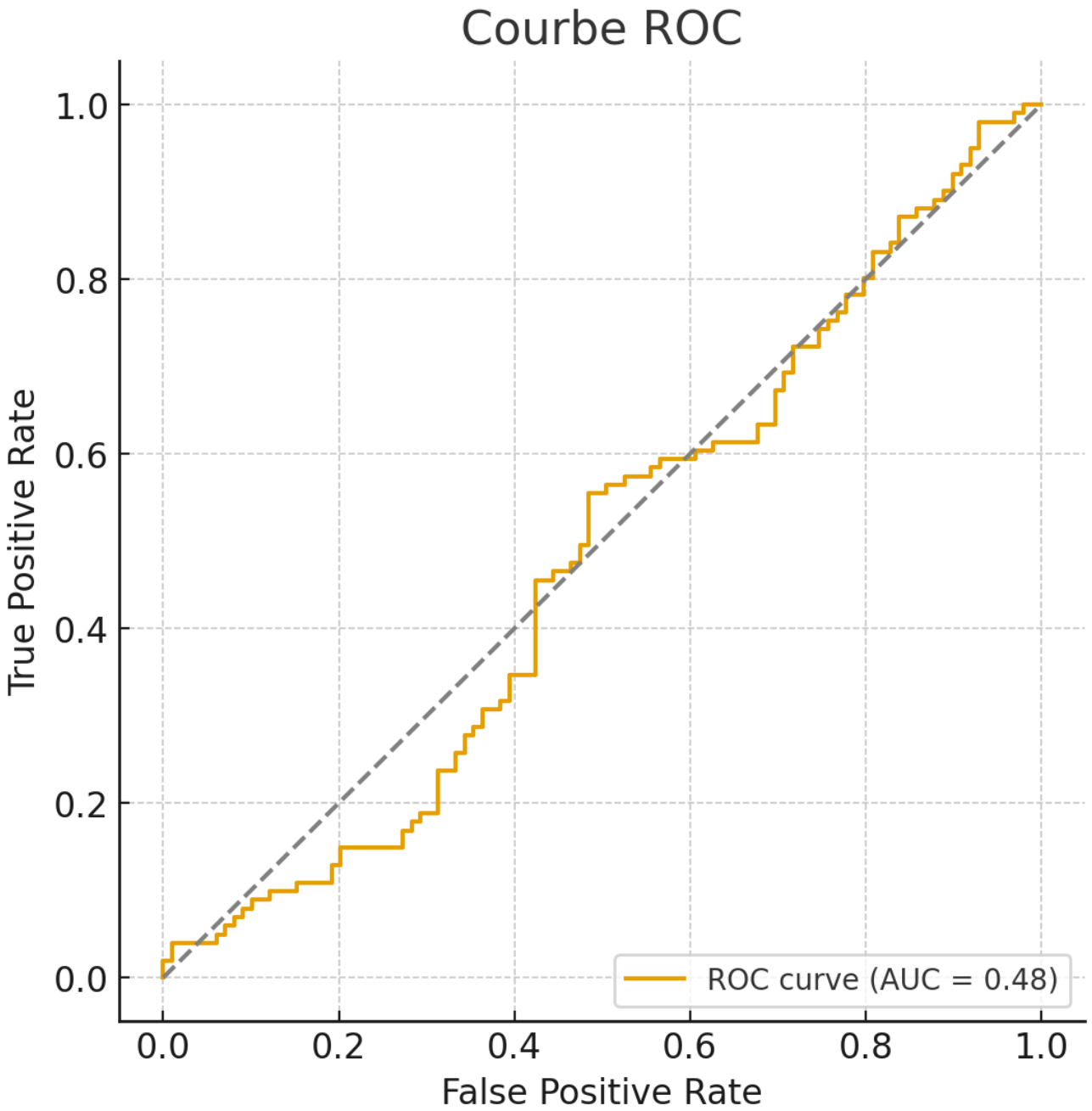
Courbe ROC et AUC

- ROC : **True Positive Rate (TPR)** vs **False Positive Rate (FPR)** en faisant varier le seuil.

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

- AUC = aire sous la courbe : probabilité qu'un positif ait un score supérieur à un négatif.
- Interprétation :
  - AUC = 0.5 : aléatoire.
  - AUC proche de 1 : très bon modèle.

Exemple de courbes ROC



Plan

1. Introduction

2. Les données : le cœur du ML

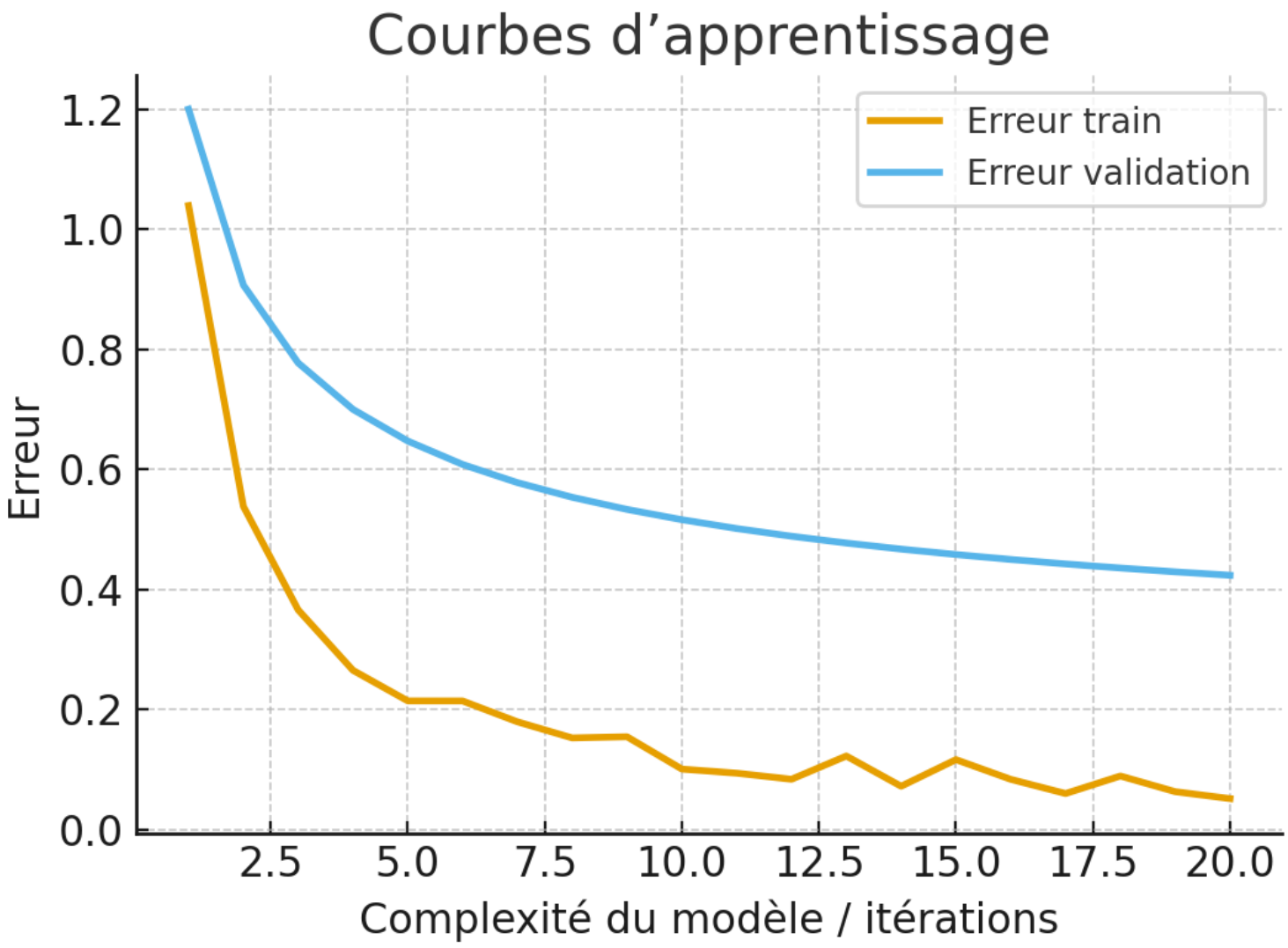
3. Algorithmes fondamentaux

4. Évaluation et sélection de modèle
4.1 Métriques principales
4.2 Biais/variance

Sur-apprentissage vs sous-apprentissage

- **Sous-apprentissage (high bias)** : modèle trop simple.
- **Sur-apprentissage (high variance)** : modèle trop complexe.
- Solutions : régularisation, early stopping, validation croisée, plus de données.

Courbes d'apprentissage



Ressources supplémentaires